

Agent 架构设计教程

从 ReAct 循环到 Claude Code、Codex、OpenClaw
与生产级多智能体系统

本教程围绕“如何选择、组合并评审 Agent 架构”展开，覆盖：

- 反应式、有限状态机、行为树、BDI、HTN 等经典智能体架构；
- ReAct、CodeAct、Plan-and-Execute、ReWOO、LLMCompiler、树搜索、反思与记忆中心架构；
- Router、Handoff、Supervisor-Workers、Debate、SOP、Blackboard、Swarm 等多智能体拓扑；
- Claude Code、Codex、OpenClaw、OpenHands、SWE-agent、Aider 等代表性系统的分层拆解；
- 研究、浏览器、数据分析、安全、具身、客服和长期个人助理等功能型 Agent；
- 运行时、协议、安全、评测、可观测性与生产级架构选型方法。

版本：2026-08-02

基于截至该日期可公开获取的产品文档、开源代码与研究论文整理

阅读说明

本书把 Agent 看作一个受约束的闭环软件系统，而不是“套了一段系统提示词的聊天模型”。重点不在于罗列框架 API，而在于建立可以迁移到不同产品、不同模型和不同业务的架构思维。

风险提示

Claude Code、Codex 等商业产品的完整内部实现并未全部公开。本书只把官方文档、公开 SDK、开源仓库和论文能够确认的行为描述为“已知机制”；涉及未公开实现时，只给出教学用的抽象模型，并明确标注为推断或设计建议。

推荐三种阅读路径：

- **建立完整架构谱系**：先读第一篇与第二篇，再读多智能体拓扑和综合选型章节。
- **重点研究 Coding Agent**：先掌握工具循环、规划、验证和耐久工作流，再阅读 Claude Code、Codex 与开源 Coding Agent 章节。
- **准备系统设计或落地项目**：重点阅读协议与运行时、安全、评测、生产化以及案例选型章节。

目录

阅读说明	i
第一章 Agent 到底是什么	1
1.1 从“回答问题”到“改变环境”	1
1.2 一个统一的数学抽象	1
1.3 Agent 的三层结构	2
第二章 Agent 核心循环与控制权	3
2.1 最小闭环	3
2.2 模型主导与代码主导	3
2.3 停止条件	4
第三章 上下文、状态与记忆	5
3.1 上下文不是记忆	5
3.2 上下文预算	5
3.3 四类记忆	5
3.4 记忆写入策略	6
第四章 工具、动作空间与环境接口	7
4.1 三种动作空间	7
4.1.1 结构化函数调用	7
4.1.2 Shell 与文件操作	7
4.1.3 CodeAct	7
4.2 Agent-Computer Interface	7
4.3 工具描述设计	8
4.4 MCP 与外部能力	8
第一部分 单智能体架构谱系	9
第五章 经典智能体架构：反应式、状态机、行为树、BDI 与 HTN	10
5.1 为什么 LLM Agent 仍需要经典架构	10
5.2 反应式架构：Sense-Act	10
5.2.1 核心思想	10

5.2.2 运行流程与模块	10
5.2.3 适用场景	11
5.2.4 优缺点与设计要点	11
5.3 有限状态机 (FSM)	11
5.3.1 核心思想	11
5.3.2 运行流程	11
5.3.3 适用场景	11
5.3.4 优缺点与设计要点	11
5.4 行为树 (Behavior Tree)	12
5.4.1 执行语义	12
5.4.2 与 LLM 的结合	12
5.4.3 适用场景与局限	12
5.5 BDI: Belief-Desire-Intention	12
5.5.1 三类状态	12
5.5.2 典型循环	12
5.5.3 LLM 化设计	13
5.5.4 适用场景与局限	13
5.6 HTN: 层次任务网络	13
5.6.1 架构组成	13
5.6.2 与 Plan-and-Execute 的差异	13
5.6.3 适用场景	13
5.7 五种经典架构的对比	13
第六章 ReAct、工具路由与 CodeAct 架构	15
6.1 ReAct: 推理与行动交替	15
6.1.1 状态与控制权	15
6.1.2 优势	15
6.1.3 局限	15
6.2 工具路由与 MRKL 风格架构	16
6.2.1 三层设计	16
6.2.2 工具发现	16
6.3 Act-only 与函数调用循环	16
6.4 CodeAct: 以可执行代码作为动作	16
6.4.1 架构组成	16
6.4.2 适用场景	16

6.4.3 安全与可靠性	17
6.5 错误恢复模式	17
第七章 显式规划、ReWOO 与编译式 DAG 执行	18
7.1 Plan-and-Execute	18
7.1.1 计划对象	18
7.1.2 执行策略	18
7.1.3 优缺点	18
7.2 ReWOO: 规划与观测解耦	18
7.2.1 关键机制	18
7.2.2 适用场景	19
7.3 LLMCompiler: 把工具编排当作编译问题	19
7.3.1 设计价值	19
7.3.2 工程难点	19
7.4 LLM 与 HTN 的混合规划	19
7.5 计划修复而非计划重写	20
第八章 树搜索、候选探索与 LATS	21
8.1 为什么需要搜索式 Agent	21
8.2 Tree of Thoughts	21
8.2.1 节点与扩展	21
8.2.2 适用性	21
8.3 Best-first、Beam Search 与候选池	21
8.4 LATS: 语言 Agent 的蒙特卡洛树搜索	21
8.4.1 四个关键环节	22
8.4.2 优势与代价	22
8.5 世界模型与价值函数	22
8.6 搜索预算设计	22
第九章 自我改进、Critic 与外部验证架构	23
9.1 Self-Refine: 同一模型的反馈循环	23
9.1.1 运行流程	23
9.1.2 主要风险	23
9.2 Reflexion: 把失败转化为情景记忆	23
9.2.1 记忆结构	23
9.2.2 适用场景	23

9.3 CRITIC: 工具交互式批评	24
9.3.1 架构拆分	24
9.4 Evaluator-Optimizer	24
9.4.1 何时适用	24
9.5 Generator-Verifier-Repair 三段式	24
9.6 确定性验证优先	24
第十章 记忆中心 Agent: RAG、MemGPT、Generative Agents 与技能库	26
10.1 上下文、状态与记忆的区别	26
10.2 检索增强 Agent	26
10.2.1 Agentic RAG 与单次 RAG	26
10.2.2 设计原则	26
10.3 MemGPT/Letta: 虚拟上下文管理	26
10.3.1 分层结构	26
10.3.2 优势与风险	27
10.4 Generative Agents: 记忆流、反思与计划	27
10.4.1 架构启示	27
10.4.2 适用场景	27
10.5 Voyager: 自动课程与可执行技能库	27
10.5.1 技能库机制	27
10.5.2 对通用 Agent 的启示	27
10.6 记忆写入策略	27
10.7 遗忘与压缩	28
第十一章 图 workflow、状态机、事件驱动与耐久执行	29
11.1 顺序 workflow	29
11.2 条件图与循环图	29
11.2.1 状态设计	29
11.2.2 路由设计	29
11.3 DAG 与数据流架构	29
11.4 事件驱动 Agent	30
11.4.1 优势	30
11.4.2 难点	30
11.5 耐久执行	30
11.6 Human-in-the-Loop	30

第十二章 黑板与共享工作空间架构	31
12.1 经典黑板架构	31
12.2 黑板中的信息类型	31
12.3 控制策略	31
12.4 适用场景	32
12.5 与群聊式多 Agent 的差异	32
第十三章 单智能体混合架构与选型	33
13.1 常见混合方式	33
13.2 六个选型维度	33
13.3 选型矩阵	33
第二部分 多智能体架构与协作拓扑	35
第十四章 多智能体系统的基本问题	36
14.1 为什么拆成多个 Agent	36
14.2 五种拓扑维度	36
14.3 Agent、工具与子工作流的边界	36
14.4 通信契约	36
第十五章 路由、Handoff 与 Swarm	37
15.1 中心化 Router-Specialist	37
15.1.1 路由策略	37
15.1.2 优缺点	37
15.2 Handoff 网络	37
15.2.1 共享与隔离	37
15.2.2 防止乒乓	37
15.3 去中心化 Swarm	38
15.3.1 适用场景	38
15.3.2 风险	38
第十六章 Supervisor-Workers 与 Orchestrator 架构	39
16.1 基本结构	39
16.2 静态与动态分解	39
16.3 Map-Reduce 型并行	39
16.4 Anthropic Research 的 Orchestrator-Worker	40

16.5 Magentic-One	40
16.6 Supervisor 的失败模式	40
第十七章 群聊、角色扮演与动态发言者选择	41
17.1 多 Agent Conversation	41
17.2 Round-Robin	41
17.3 Selector Group Chat	41
17.4 CAMEL 角色扮演	41
17.4.1 局限	41
17.5 对话去幻觉与共识	42
第十八章 SOP、流水线与组织模拟：MetaGPT 和 ChatDev	43
18.1 SOP 驱动的多 Agent	43
18.2 MetaGPT	43
18.2.1 架构要点	43
18.3 ChatDev	43
18.3.1 优缺点	43
18.4 流水线架构的通用化	44
第十九章 Debate、Committee、Ensemble 与 Judge	45
19.1 多 Agent Debate	45
19.1.1 独立性要求	45
19.1.2 终止与裁决	45
19.2 Committee 与投票	45
19.3 Mixture-of-Agents	45
19.4 Generator-Critic Committee	46
19.5 何时不要使用	46
第二十章 黑板多 Agent、Agent-as-Tool 与市场式协作	47
20.1 多 Agent 黑板	47
20.2 Agent-as-Tool	47
20.2.1 与 Handoff 的区别	47
20.3 Contract Net 与竞标	47
20.3.1 适用场景	47
20.4 市场式与效用优化	48
第二十一章 多 Agent 失败模式与选型准则	49

21.1 协调开销	49
21.2 幻觉传播	49
21.3 共享上下文污染	49
21.4 写冲突与并行副作用	49
21.5 责任与身份	49
21.6 使用多 Agent 的判断问题	49
第三部分 代表性产品与功能型 Agent 架构	51
第二十二章 Coding Agent 的通用架构	52
22.1 Coding Agent 的任务闭环	52
22.2 任务契约	52
22.3 仓库理解层	52
22.4 编辑层	52
22.5 验证层	52
22.6 环境与权限	53
22.7 长任务架构	53
22.8 子 Agent 的合理使用	53
第二十三章 Claude Code 架构详解	54
23.1 公开可确认的核心循环	54
23.2 上下文层	54
23.3 工具循环	54
23.4 Hooks: 生命周期上的确定性控制	54
23.4.1 Hook 与 Prompt 的边界	55
23.5 Subagents	55
23.6 Skills、Plugins 与 MCP	55
23.7 记忆与压缩	55
23.8 权限与沙箱	55
23.9 长任务 Harness	55
23.10 适用场景与局限	55
第二十四章 Codex 架构详解	56
24.1 多表面与多环境	56
24.2 Local、Worktree 与 Cloud	56
24.3 AGENTS.md 层次化指令	56

24.4 Skills 与定制层	56
24.5 Subagents 与并行线程	56
24.6 沙箱与审批	57
24.7 自动化与长程任务	57
24.8 多 Agent 控制台思路	57
24.9 与 Claude Code 的架构比较	57
第二十五章 开源 Coding Agent 架构	58
25.1 SWE-agent: ACI 优先	58
25.1.1 设计启示	58
25.2 OpenHands: 事件流与 CodeAct	58
25.2.1 平台化意义	58
25.3 Aider: Repository Map 与 Architect/Editor 分离	58
25.4 Agentless: 确定性流水线基线	59
25.5 MASAI: 模块化软件工程 Agent	59
25.6 五种系统的对比	59
第二十六章 OpenClaw: 网关、会话、记忆与长期运行架构	60
26.1 定位	60
26.2 Gateway 控制平面	60
26.3 每会话串行 Agent Loop	60
26.4 Workspace 与 Memory	60
26.5 Skills 与 Subagents	61
26.6 多 Agent 路由	61
26.7 Automations	61
26.8 Runtime 与 Harness	61
26.9 安全边界	61
26.10 适用与不适用	61
第二十七章 研究与深度检索 Agent	62
27.1 典型分层	62
27.2 问题分解	62
27.3 并行搜索 Worker	62
27.4 阅读与证据抽取	62
27.5 综合与冲突	62
27.6 引用验证	62

27.7 架构选择	63
第二十八章 浏览器与 Computer-use Agent	64
28.1 感知-决策-动作闭环	64
28.2 DOM/Accessibility Tree 架构	64
28.3 纯视觉 Computer-use	64
28.4 混合感知	64
28.5 任务规划	64
28.6 安全问题	64
第二十九章 数据分析与 SQL Agent	65
29.1 架构目标	65
29.2 语义层	65
29.3 Planner-Generator-Executor-Verifier	65
29.4 查询修复	65
29.5 权限与隐私	65
29.6 多 Agent 设计	65
第三十章 安全与漏洞分析 Agent	66
30.1 分层流水线	66
30.2 知识与模式层	66
30.3 静态与动态协同	66
30.4 Harness	66
30.5 Memory 与反作弊	66
30.6 权限和隔离	66
30.7 适合的多 Agent 拓扑	67
第三十一章 具身、游戏与机器人 Agent	68
31.1 层次控制	68
31.2 感知与世界状态	68
31.3 技能库	68
31.4 行为树与 LLM 的结合	68
31.5 仿真与现实	68
第三十二章 客服、个人助理与长期运行 Agent	69
32.1 客服 Agent	69
32.1.1 交易型与解释型任务	69

32.2 个人助理	69
32.3 长期运行	69
32.4 主动性控制	69
32.5 OpenClaw 式 Gateway 的价值	69
第四部分 框架、运行时与生产级治理	70
第三十三章 Agent 框架与协议的架构定位	71
33.1 不要把框架当作架构	71
33.2 OpenAI Agents SDK	71
33.3 Claude Agent SDK	71
33.4 LangGraph	71
33.5 Google ADK	71
33.6 AutoGen	72
33.7 CrewAI	72
33.8 MCP 与 A2A	72
33.8.1 协议不等于信任	72
33.9 框架选型表	72
第三十四章 长任务、队列、检查点与故障恢复	74
34.1 任务持久化	74
34.2 队列与 Worker	74
34.3 检查点	74
34.4 幂等与副作用	74
34.5 重试策略	74
34.6 取消与超时	74
34.7 跨上下文窗口连续性	75
第三十五章 Agent 安全架构与威胁模型	76
35.1 核心风险	76
35.2 零信任分层	76
35.3 Prompt Injection 防护	76
35.4 工具安全	76
35.5 凭据与身份	77
35.6 沙箱	77
35.7 审批设计	77

35.8 安全验证	77
第三十六章 评测、Tracing 与调试	78
36.1 三层评测	78
36.2 结果指标	78
36.3 轨迹指标	78
36.4 Tracing	78
36.5 失败分类	78
36.6 消融实验	78
36.7 回放与确定性	79
36.8 Agent Evals 的特性	79
第三十七章 架构选型案例与设计审查	80
37.1 案例一：内部知识问答	80
37.2 案例二：自动修复代码 Issue	80
37.3 案例三：跨来源研究报告	80
37.4 案例四：企业客服与业务办理	80
37.5 案例五：持续安全漏洞分析	80
37.6 案例六：个人长期助理	81
37.7 设计审查的十二个问题	81
附录 A 架构术语速查	82
附录 B 架构模式总表	83
附录 C 进一步阅读路线	84
C.1 经典控制与规划	84
C.2 LLM 单 Agent	84
C.3 多 Agent	84
C.4 Coding Agent	84
参考资料	85

Agent 到底是什么

1.1 从“回答问题”到“改变环境”

普通聊天模型的输入和输出都停留在文本空间。Agent 的关键差异是：模型可以读取外部状态、选择动作、调用工具、观察执行结果，并根据结果继续决策。因而，一个 Agent 至少包含以下五类要素：

1. **目标**：用户要求、系统任务或持续性职责；
2. **状态**：当前上下文、历史、工作区、任务进度与环境状态；
3. **策略**：模型和控制程序共同决定下一步动作；
4. **动作空间**：函数调用、文件操作、Shell、浏览器、GUI、消息发送等；
5. **反馈**：工具输出、测试结果、用户审批、环境奖励与验证器结论。

定义

在工程上，可以把 Agent 定义为：一个以大模型为策略核心、以工具和环境为动作接口、以状态与反馈为闭环，并受到权限、预算和停止条件约束的长期或短期执行系统。

1.2 一个统一的数学抽象

令第 t 步的用户目标为 g ，会话历史为 h_t ，长期记忆为 m_t ，环境观测为 o_t ，可用工具描述为 T_t ，系统策略与权限为 p_t 。上下文构造器把这些信息压缩为模型可见上下文：

$$c_t = B(g, h_t, m_t, o_t, T_t, p_t), \quad (1.1)$$

其中 $B(\cdot)$ 是上下文组装与裁剪函数。模型策略产生自然语言输出、动作或停止信号：

$$(r_t, a_t, z_t) \sim \pi_\theta(\cdot \mid c_t), \quad (1.2)$$

其中 r_t 表示可见回复或内部计划， a_t 表示工具动作， $z_t \in \{\text{continue, stop, escalate}\}$ 表示控制状态。环境执行动作后返回：

$$o_{t+1}, e_{t+1} = E(a_t), \quad (1.3)$$

其中 e_{t+1} 是错误、测试结果或奖励信号。状态管理器再更新历史和记忆：

$$(h_{t+1}, m_{t+1}) = U(h_t, m_t, a_t, o_{t+1}, e_{t+1}). \quad (1.4)$$

关键结论

模型只负责“提出下一步”，真正决定系统是否可靠的是：上下文如何构造、动作如何约束、结果如何验证、失败如何恢复、何时停止。

1.3 Agent 的三层结构

一个实用的心智模型是把 Agent 分成三层：



图 1.1: Agent 的三层结构。模型只是最上层的一部分。

认知层决定“想做什么”，运行时决定“是否允许以及怎样执行”，环境层决定“实际上能做什么”。把三层混在一段 Prompt 中，是许多原型系统难以扩展和审计的根本原因。

Agent 核心循环与控制权

2.1 最小闭环

最常见的 Agent Loop 可以表示为：

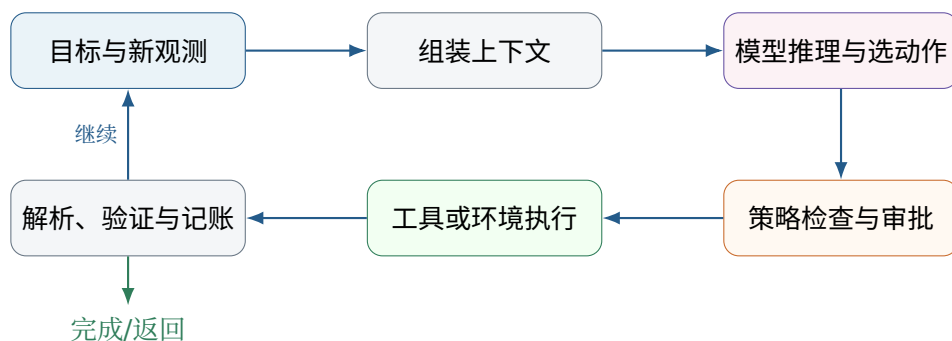


图 2.1：一个可审计的 Agent Loop。

模型的每一次输出都应被视为一个**提案**。运行时必须完成结构化解析、工具存在性检查、参数校验、权限判断、速率限制和风险审批，然后才能执行。

2.2 模型主导与代码主导

Agent 架构最重要的选择之一，是把控制权交给模型还是程序。

维度	模型主导型	代码主导型
流程决定	模型自行选择下一步与工具	代码定义固定节点、路由和循环
灵活性	高，适合开放任务	中等，适合已知流程
可预测性	较低	较高
调试方式	看轨迹与提示词	看状态机、日志和节点输入输出
典型形态	ReAct、Coding Agent、个人助理	DAG、审批流、ETL、报告流水线
主要风险	漫游、过度调用、权限误用	流程僵化、异常分支覆盖不足

生产系统通常采用**混合控制**：代码规定阶段、权限与停止条件，模型只在局部节点内做开放决策。例如：代码先固定“理解问题 → 制定计划 → 修改代码 → 测试 → 生成报告”，但每个阶段内部允许模型多轮探索。

2.3 停止条件

没有停止条件的 Agent 会陷入重复搜索、反复修改或无意义的自我讨论。常用停止条件包括：

- 任务验证器通过，例如全部目标测试通过；
- 模型返回结构化完成信号，并且运行时验证充分；
- 达到最大步数、Token、时间或费用预算；
- 连续 k 步没有状态增益；
- 相同错误或动作重复超过阈值；
- 需要越权、不可逆操作或缺少关键输入，转人工处理。

可以定义进展函数 Δ_t ：

$$\Delta_t = S(s_{t+1}) - S(s_t), \quad (2.1)$$

其中 S 是任务状态评分。如果连续多步 $\Delta_t \leq 0$ ，系统应触发重规划、降级或停止，而不是继续消耗预算。

上下文、状态与记忆

3.1 上下文不是记忆

上下文是当前一次模型调用实际可见的信息；**记忆**是可以跨调用、跨会话或跨任务保存的信息。记忆只有在被检索并注入上下文后才会影响模型行为。

上下文通常由以下部分组成：

- 系统规则与角色；
- 项目级说明文件，如 `CLAUDE.md` 或 `AGENTS.md`；
- 工具描述、权限和环境信息；
- 当前会话消息与工具结果；
- 按需检索的文件、文档和长期记忆；
- 当前计划、任务清单和状态摘要。

3.2 上下文预算

设上下文窗口上限为 L ，系统规则、工具模式、会话、检索材料、工作记忆分别占用 l_s, l_t, l_h, l_r, l_w ，则必须满足：

$$l_s + l_t + l_h + l_r + l_w + l_{\text{reserve}} \leq L, \quad (3.1)$$

其中 l_{reserve} 为模型输出、工具结果和突发情况预留空间。

设计建议

不要把整个仓库、全部历史和所有技能一次性塞入上下文。更好的策略是：启动时只加载稳定且高价值的“热上下文”，其余信息通过搜索、索引和工具按需获取。

3.3 四类记忆

1. **工作记忆**：当前任务计划、待办、最近工具输出；
2. **情景记忆**：以前尝试过什么、为何失败、用户曾如何纠正；
3. **语义记忆**：项目架构、领域知识、用户长期偏好；
4. **程序性记忆**：可复用技能、脚本、操作流程和策略模板。

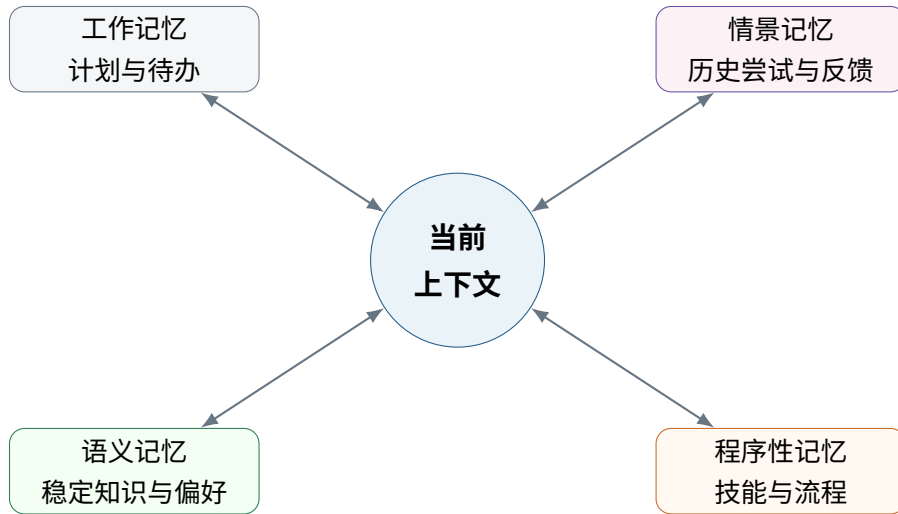


图 3.1: 记忆需要通过检索、摘要或显式加载进入当前上下文。

3.4 记忆写入策略

“让模型自动记住一切”通常会造成污染。更稳健的写入流程是：

1. 提取候选记忆；
2. 判断是否长期有效、是否与现有内容冲突；
3. 加上来源、时间和置信度；
4. 对高风险记忆请求人工确认；
5. 支持删除、过期、版本化和回滚。

对于 Coding Agent，最有价值的记忆常常不是聊天摘要，而是“构建命令、测试命令、架构约束、常见失败、目录职责、禁止修改区域”等可操作信息。

工具、动作空间与环境接口

4.1 三种动作空间

4.1.1 结构化函数调用

模型选择预定义工具并生成 JSON 参数。优点是可校验、可授权、容易观测；缺点是组合能力受工具粒度限制。

4.1.2 Shell 与文件操作

Coding Agent 常直接使用读取、搜索、编辑、执行命令等工具。它们能力强，但风险也高。一个 Shell 工具实际上可以间接完成文件读写、网络访问、进程控制和依赖安装，因此必须把它视为高权限入口。

4.1.3 CodeAct

CodeAct 把可执行代码作为统一动作空间，让模型通过 Python 等语言组合 API、数据结构和控制流。相关研究表明，可执行代码动作可以提高复杂工具组合能力 [3]。代价是必须提供严格沙箱、依赖限制、超时和资源控制。

4.2 Agent-Computer Interface

SWE-agent 强调 **Agent-Computer Interface, ACI** 的重要性：并非给模型一个普通终端就够了，接口的命令设计、输出格式、编辑方式和信息密度会显著影响 Agent 的表现 [34]。

一个好的 ACI 应满足：

- 反馈短而结构化，不让无关日志淹没上下文；
- 文件定位和编辑具有稳定语义；
- 支持查看差异、撤销和局部修改；
- 命令失败时给出明确错误与下一步线索；
- 工具返回结果包含来源、截断信息和执行状态；
- 同一动作在不同环境中行为尽量一致。

4.3 工具描述设计

工具描述既是 API 文档，也是模型的“动作先验”。应包含：

- 何时使用与何时不要使用；
- 参数语义、边界、默认值；
- 是否有副作用和不可逆性；
- 可能的错误与重试方式；
- 返回值的结构和截断策略；
- 权限与审批要求。

工程示例

一个名为 `run_tests` 的高层工具通常比开放的 `bash("pytest ...")` 更可控，因为它可以固定超时、并发数、环境变量、日志截断、失败解析和缓存策略。底层 Shell 仍可保留，但仅在高层工具无法覆盖时启用。

4.4 MCP 与外部能力

Model Context Protocol(MCP)把工具和数据源以统一协议暴露给 Agent。Claude Code、Codex 等产品都支持 MCP 形式的扩展 [44, 52]。MCP 的价值不只是“多接几个工具”，而是形成清晰的能力边界：服务器负责认证、数据访问与业务逻辑，Agent 只看到经过描述和授权的工具。

需要注意：MCP 服务器本身仍是受信任代码。它可能泄露数据、返回恶意内容或提供过宽权限，因此必须进行来源审核、最小权限配置和输出净化。

第一部分

单智能体架构谱系

经典智能体架构：反应式、状态机、行为树、BDI 与 HTN

5.1 为什么 LLM Agent 仍需要经典架构

LLM 带来了自然语言理解、开放式规划和工具选择能力，但没有消除控制问题。只要系统需要持续运行、处理失败、满足实时约束或保证关键动作顺序，就仍然要回答：状态由谁维护、动作何时触发、计划何时失效、异常如何恢复、哪些行为必须确定性执行。经典智能体架构提供了这些问题的清晰术语和成熟结构。

关键结论

LLM 最适合承担语义理解、候选生成和不完备信息下的判断；状态迁移、权限边界、超时、重试、验收和安全联锁通常应由确定性控制层承担。现代 Agent 往往不是“经典架构或 LLM”的二选一，而是经典控制骨架中嵌入 LLM 节点。

5.2 反应式架构：Sense-Act

5.2.1 核心思想

反应式 Agent 根据当前观测直接选择动作，不显式维护长期计划。其抽象形式为

$$a_t = \pi(o_t),$$

其中 o_t 是当前观测， a_t 是动作。最简单的规则是“看到条件 c 就执行动作 a ”；更复杂的实现会把多个行为模块按优先级叠加。

5.2.2 运行流程与模块

系统通常包含感知适配器、特征或事件提取器、规则/策略选择器、动作执行器和安全联锁。每轮只处理最新环境状态，旧观测可以被丢弃或仅用于低层滤波。由于没有复杂计划，响应延迟低，失效模式也较容易分析。

5.2.3 适用场景

反应式架构适合告警处理、简单运维处置、游戏 NPC、机器人避障、固定格式客服分流、工具调用前的安全拦截，以及任何“输入条件明确、动作空间小、时效性高”的任务。

5.2.4 优缺点与设计要点

优势是快速、可预测、易测试；不足是缺乏长程目标、容易在局部规则间振荡，也无法处理需要跨步骤证据的任务。使用 LLM 时，应把它限制为分类器或参数提取器，并为低置信度设置降级分支。不要让模型在高频控制环中自由生成任意动作。

5.3 有限状态机 (FSM)

5.3.1 核心思想

FSM 把系统表示为有限状态集合、事件集合和状态迁移函数。每个状态对应一组允许行为，只有满足显式条件才迁移到下一状态。典型 Agent 可包含 *Idle*、*Planning*、*Executing*、*WaitingApproval*、*Verifying*、*Succeeded*、*Failed* 等状态。

5.3.2 运行流程

输入事件先经过规范化，再由当前状态与迁移表决定下一状态及副作用。状态是系统事实来源，模型不能用自然语言声称“已经完成”来绕过迁移条件。失败、超时和取消都应建模为一等事件。

5.3.3 适用场景

FSM 适合审批流、订单处理、部署发布、表单助手、设备操作、账户恢复、支付和任何阶段边界清楚的 Agent。它也适合包裹一个自由度较高的 LLM Loop：模型在 *Executing* 状态内探索，但必须通过验证器才能进入 *Succeeded*。

5.3.4 优缺点与设计要点

FSM 的优势是可视化、可审计、便于覆盖测试；缺点是状态数量容易爆炸，跨状态共享逻辑会变得重复。应将“业务阶段”建模为状态，把可复用动作放入独立节点或子状态机，并避免为每个微小错误创建新状态。

5.4 行为树 (Behavior Tree)

行为树把任务切换逻辑组织为一棵层次树，常见控制节点包括 **Sequence**、**Selector/Fall-back**、**Parallel** 和装饰器；叶节点是条件检查或动作。行为树最初广泛用于游戏和机器人，其模块化、反应性和复用性优于大型 FSM[16, 17]。

5.4.1 执行语义

树从根节点周期性 Tick。**Sequence** 要求子节点依次成功；**Selector** 在前一个子节点失败后尝试后续节点；**Parallel** 根据阈值聚合多个分支；装饰器可实现重试、超时、取反和限流。每个节点返回 **Success**、**Failure** 或 **Running**。

5.4.2 与 LLM 的结合

LLM 可以作为叶节点完成开放式判断，也可以生成候选子树供控制器审核。关键动作仍由确定性节点执行。例如“修复服务”可设计为：检查健康状态 → 尝试轻量恢复 → 验证 → 若失败则请求审批后重启。模型只负责诊断和选择参数，不控制安全顺序。

5.4.3 适用场景与局限

行为树适合需要持续反应、回退策略和模块复用的机器人、GUI 操作、运维和游戏 Agent。它比 FSM 更容易组合，但对复杂数据依赖、全局计划和长时持久化支持有限，通常需要外部黑板或状态存储。

5.5 BDI: Belief-Desire-Intention

BDI 把 Agent 的内部状态分为信念、愿望和意图，是经典实践推理模型之一 [14, 15]。

5.5.1 三类状态

Belief 表示 Agent 对世界的当前认知，允许不完整甚至错误；**Desire** 表示候选目标；**Intention** 表示已经承诺并正在执行的目标或计划。核心价值在于区分“想做什么”与“已经决定投入资源做什么”。

5.5.2 典型循环

系统接收事件并修订信念，从愿望中筛选可行目标，形成或更新意图，选择匹配计划，执行一步动作，再根据结果判断是否继续承诺、暂停或放弃。与普通 ReAct 相比，BDI 对目标承诺和计划失效有更显式的表示。

5.5.3 LLM 化设计

可以用结构化对象保存信念来源、时间戳和置信度；用规则或模型生成候选愿望；用成本、风险、优先级和依赖约束决定意图。LLM 可解释冲突并提出计划，但意图提交应经过策略引擎。这样可以避免 Agent 每次收到新消息都无条件改写目标。

5.5.4 适用场景与局限

BDI 适合个人助理、持续运维、游戏角色、组织代理和需要同时管理多个目标的系统。缺点是信念一致性、意图撤销和计划库维护复杂；若所有状态仍以无结构对话历史表示，BDI 只会退化为术语包装。

5.6 HTN：层次任务网络

HTN 规划利用领域知识把抽象任务逐层分解为可执行原子任务,能够显著缩小搜索空间 [18]。一个复合任务可有多种分解方法，每种方法带前置条件、子任务关系和约束。

5.6.1 架构组成

HTN Agent 包含任务库、方法库、世界状态、分解器、约束检查器和执行器。规划器从高层目标开始，选择适用方法，递归分解，直到得到工具可直接执行的动作序列或部分有序网络。

5.6.2 与 Plan-and-Execute 的差异

普通 Plan-and-Execute 往往由模型临时生成计划；HTN 的分解方法来自经过验证的领域流程。前者灵活但不稳定，后者可靠但需要建模成本。生产系统常用混合方式：先由 HTN 固定关键骨架，再让 LLM 在某些开放节点生成具体方案。

5.6.3 适用场景

HTN 适合安全响应、数据处理、旅行规划、合规审查、设备维护、科研实验和复杂企业流程。只要“正确顺序”比“完全自由探索”更重要，HTN 往往比纯 ReAct 更合适。

5.7 五种经典架构的对比

设计建议

对现代 LLM Agent，推荐把 FSM/行为树/HTN 用作外层控制骨架，把 BDI 用作长期目标与承诺模型，把 LLM 工具循环放在开放式叶节点中。这样既保留语言模型的适应性，又不把系统可靠性寄托在单次生成上。

架构	控制依据	主要优势	主要局限
反应式	当前观测与规则	低延迟、易验证、适合高频响应	缺乏长程目标与跨步推理
FSM	显式状态与迁移	可审计、异常路径清楚	状态爆炸、组合复用较弱
行为树	层次化 Tick 与回退	模块化、反应性强、易复用	全局数据依赖与持久化需外接
BDI	信念、愿望、意图	适合多目标和持续承诺	状态维护与一致性复杂
HTN	领域任务分解方法	流程可靠、搜索空间小	领域建模成本高、灵活性有限

表 5.1: 经典智能体架构的控制特征。

ReAct、工具路由与 CodeAct 架构

6.1 ReAct：推理与行动交替

ReAct 让模型在推理、动作和观测之间循环，利用环境反馈修正后续决策 [1]。它是最常见的通用 LLM Agent 架构，因为无需预先枚举完整 workflow，就能处理搜索、代码、网页和开放式工具任务。

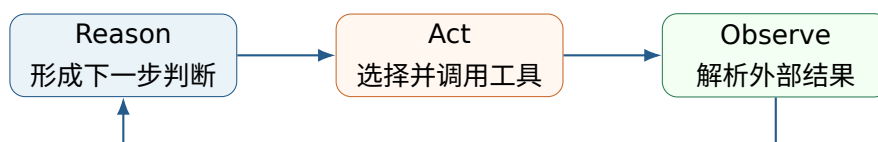


图 6.1: ReAct 的基本闭环。

6.1.1 状态与控制权

每轮输入通常包含目标、当前上下文、工具描述和历史轨迹；模型输出工具调用或最终回答。控制程序负责执行工具、校验参数、捕获错误、追加观测并判断是否继续。模型决定“下一步做什么”，代码决定“这个动作是否允许以及如何执行”。

6.1.2 优势

ReAct 能利用新观测动态调整策略，适合环境不确定、工具结果不可预知或问题必须逐步探索的任务。它的轨迹天然可观察，便于分析模型为何选择某个工具。

6.1.3 局限

主要问题包括循环、重复搜索、上下文增长、每步都需要模型调用、对错误观测过度反应，以及把局部进展误判为任务完成。工程上必须增加步数预算、重复动作检测、结构化结果、停止条件和验证器。

6.2 工具路由与 MRKL 风格架构

MRKL 类架构把多个专家模块或工具放在一个路由器之后，由模型判断问题应交给哪个模块 [2]。与完整 ReAct 相比，路由器通常只做一次或少量决策，适合请求类型明确、专家能力边界稳定的系统。

6.2.1 三层设计

第一层是意图与风险分类；第二层是工具/专家选择；第三层是参数抽取和结果标准化。路由决策应输出结构化标签、置信度和理由摘要。低置信度时可回退到通用 Agent、检索更多上下文或请求用户澄清。

6.2.2 工具发现

工具数量达到数百或数千时，不应把所有 Schema 塞进上下文。可以先按元数据、权限和语义检索候选工具，再让模型在小集合中选择。工具注册表应包含版本、所有者、副作用、权限、成本、延迟、幂等性和错误语义。

6.3 Act-only 与函数调用循环

并非所有 Agent 都需要显式自然语言推理。Act-only 架构让模型直接产生结构化函数调用，观测返回后再次选择动作。其优点是输出更短、协议更清晰、隐私风险较小；不足是复杂任务中缺少显式计划和错误诊断空间。

适用场景包括 API 编排、数据库查询、表单填充和结构化企业助手。设计重点是严格 Schema、参数约束、服务端授权和工具级 Guardrail，而不是要求模型“思考得更久”。

6.4 CodeAct：以可执行代码作为动作

CodeAct 使用可执行代码表达行动，使模型能通过变量、循环、条件和库调用组合多个工具 [3]。与逐个 JSON 函数调用相比，代码可把大量中间计算放在执行环境中，降低模型往返次数。

6.4.1 架构组成

系统通常包含代码生成模型、受限解释器、工具/API 绑定、文件或变量工作区、执行日志和安全策略。模型生成一段代码，运行时执行并返回 stdout、异常和结构化产物；模型再修复或继续。

6.4.2 适用场景

CodeAct 适合数据分析、科研计算、批量 API 操作、代码库修改和需要动态组合工具的任务。它不适合直接接触高权限系统，因为一段代码可以产生比单一函数调用更宽的副作用。

6.4.3 安全与可靠性

必须限制可导入模块、文件系统、网络、进程、CPU、内存和执行时间；工具对象应采用能力注入，而不是把全局凭据暴露给解释器。执行结果需要截断和结构化，否则日志会迅速耗尽上下文。

6.5 错误恢复模式

工具型 Agent 的错误可分为参数错误、环境错误、权限错误、暂时性失败、逻辑错误和不可恢复失败。推荐将错误转换为机器可读对象，包含错误类别、是否可重试、建议动作、已消耗预算和关联调用 ID。模型不应从一段模糊堆栈中自行猜测所有恢复策略。

设计建议

ReAct 适合开放探索；Router 适合稳定分流；Act-only 适合结构化事务；CodeAct 适合复杂组合。实际系统可以分层使用：先路由到领域 Agent，再由领域 Agent 在函数调用与 CodeAct 之间选择。

显式规划、ReWOO 与编译式 DAG 执行

7.1 Plan-and-Execute

Plan-and-Execute 将高层规划与低层执行分离：规划器先生成步骤，执行器逐步完成，必要时由重规划器修订剩余计划。与 ReAct 每轮局部决策不同，它在执行前建立全局任务模型。

7.1.1 计划对象

高质量计划不是自然语言列表，而是结构化任务图。每个步骤应包含目标、输入、依赖、允许工具、预期产物、验收条件、失败策略、预算和风险级别。步骤状态至少包括 Pending、Ready、Running、Blocked、Succeeded、Failed 和 Skipped。

7.1.2 执行策略

执行器只处理 Ready 节点，并将产物写入状态存储。验证器决定节点是否成功；重规划器只修改未完成部分，避免每次失败都推翻已验证结果。涉及不可逆动作时，计划必须在执行前通过审批。

7.1.3 优缺点

优势是可解释、可并行、便于审批和预算估计；缺点是初始计划可能基于错误假设，计划生成本身也有成本。适合中长任务、跨系统业务流程、研究、迁移和复杂代码改造。

7.2 ReWOO：规划与观测解耦

ReWOO 将 Planner、Worker 和 Solver 分离：Planner 预先写出包含变量依赖的推理蓝图，Worker 执行工具调用并填充证据，Solver 根据计划与证据生成最终答案 [4]。

7.2.1 关键机制

计划中的步骤通过变量引用前序结果，例如 E_1 表示第一次检索结果，后续步骤使用 E_1 生成查询。由于规划阶段不必在每个工具结果后重新调用大模型，ReWOO 可减少冗余上下文和串行

往返。

7.2.2 适用场景

它适合依赖结构可提前预测的检索、报告和数据聚合任务。若后续步骤强烈依赖未知观测、需要频繁改计划，ReWOO 会比 ReAct 更脆弱。工程上应允许 Worker 触发“计划无效”信号并回退到局部重规划。

7.3 LLMCompiler: 把工具编排当作编译问题

LLMCompiler 使用函数调用规划器生成任务依赖，Task Fetching Unit 根据依赖调度任务，Executor 并行执行可并行工具 [5]。

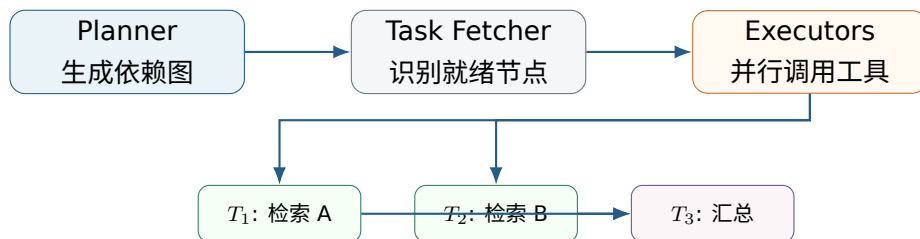


图 7.1: 编译式工具编排：先形成依赖图，再并行执行就绪节点。

7.3.1 设计价值

这类架构把“模型思考”和“系统调度”解耦。并行度、重试、缓存、超时和资源限制由执行引擎决定，而不是让模型逐步发出所有调用。它适合工具多、独立子任务多、延迟敏感的系统。

7.3.2 工程难点

Planner 可能遗漏依赖或错误地并行具有副作用的操作。任务图需要显式声明读写集合、幂等性和事务边界。对于写操作，默认应串行或使用冲突检测；对于读操作，可根据配额并行。

7.4 LLM 与 HTN 的混合规划

一种稳健方案是用 HTN 固定业务骨架，用 LLM 完成方法选择和开放参数生成。例如安全事件响应的顶层阶段固定为收集证据、评估影响、制定处置、审批、执行和复盘；每个阶段内部再允许模型调用工具探索。

7.5 计划修复而非计划重写

当一步失败时，应先判定失败类型：前置条件不满足、工具暂时失败、产物不合格、依赖错误或目标变化。只有计划假设被推翻时才重规划；否则应重试、替换工具、缩小步骤或请求输入。计划版本需要持久化，便于审计为什么路线发生改变。

树搜索、候选探索与 LATS

8.1 为什么需要搜索式 Agent

ReAct 和 Plan-and-Execute 通常沿一条轨迹前进。一旦早期决策错误，后续动作会在错误分支上继续累积。搜索式架构同时保留多个候选状态，通过评价函数选择更有前景的分支，适合高不确定、可回滚且验证信号较强的任务。

8.2 Tree of Thoughts

Tree of Thoughts 将中间“思路”作为搜索节点，通过生成、评价和选择多个候选路径实现更审慎的推理 [6]。它可使用广度优先、深度优先或束搜索。

8.2.1 节点与扩展

节点应表示可验证的部分状态，而不只是随意文字。扩展器生成若干后继；评价器估计可行性、质量或剩余成本；选择器保留 top- k 分支；终止器判断是否达到目标。

8.2.2 适用性

适合谜题、规划、代码候选、复杂写作结构和多个方案权衡。若环境动作不可逆、节点复制成本高或没有可靠评价器，树搜索收益会迅速下降。

8.3 Best-first、Beam Search 与候选池

工程系统常不需要完整树。可以维护固定大小候选池，每轮扩展分数最高的若干状态，并通过去重避免语义相同的分支。评分可综合硬约束、测试通过数、风险、成本、模型置信度和新颖性。

8.4 LATS：语言 Agent 的蒙特卡洛树搜索

LATS 将推理、行动、环境反馈、自反思和蒙特卡洛树搜索结合，使用模型作为策略和值函数的一部分 [7]。

8.4.1 四个关键环节

选择阶段根据探索/利用准则选择节点；扩展阶段生成候选动作；模拟或执行阶段从环境获得反馈；回传阶段更新路径价值。失败轨迹可产生反思，影响后续节点扩展。

8.4.2 优势与代价

LATS 能从失败分支中学习并回到较早决策点，比线性重试更强；但模型调用、环境复制和状态存储成本很高。它更适合有明确奖励、可创建沙箱快照、单次任务价值高的场景，如代码修复、网页任务和复杂交互式推理。

8.5 世界模型与价值函数

搜索架构的瓶颈通常不是生成候选，而是评价。价值函数可以是确定性测试、规则、轻量模型、强模型 Judge 或多信号融合。模型自评分容易过度自信，必须用外部证据校准。若能建立环境模拟器或世界模型，可在执行真实高风险动作前先评估后果。

8.6 搜索预算设计

应限制最大节点数、最大深度、并行宽度、单节点工具成本和总时间。搜索过程需要去重、剪枝、失败缓存和早停。对生产系统，推荐先使用“小宽度候选池 + 强验证器”，只有基准显示明显收益时再引入完整 MCTS。

自我改进、Critic 与外部验证架构

9.1 Self-Refine：同一模型的反馈循环

Self-Refine 让模型先生成结果，再对结果提供反馈并据此修订，多轮迭代无需额外训练 [8]。它适合文本质量、代码可读性、计划完整性和格式改进。

9.1.1 运行流程

生成器输出候选；反馈器按维度指出问题；修订器只针对反馈修改；停止器根据质量阈值、无新增问题或迭代预算终止。三种角色可以由同一模型不同提示承担，也可使用不同模型。

9.1.2 主要风险

若反馈没有外部依据，模型可能把正确内容改错，或在风格上反复震荡。应固定评价 Rubric，保存版本 Diff，并要求反馈引用具体证据。

9.2 Reflexion：把失败转化为情景记忆

Reflexion 将环境反馈总结成文字反思并写入记忆，使后续尝试避免重复错误 [9]。它不仅修改当前答案，还改变下一次试验的上下文。

9.2.1 记忆结构

反思应包含任务特征、失败动作、根因、可迁移规则、适用边界和证据。不可把完整长轨迹直接写入长期记忆，否则错误经验和偶然现象会污染未来任务。

9.2.2 适用场景

适合重复试验、代码调试、游戏、Web 任务和安全分析。若任务只执行一次或失败信号不可靠，Reflexion 的价值有限。

9.3 CRITIC：工具交互式批评

CRITIC 强调用搜索、解释器、代码执行等外部工具验证输出，再据此修正 [10]。它把“自我批评”从纯语言判断升级为证据驱动的检验。

9.3.1 架构拆分

Critic 先把候选分解为可检查主张，选择合适工具获得证据，比较主张与证据，生成结构化缺陷，再交给修订器。不同主张应使用不同验证器：事实用检索，计算用执行器，代码用测试，格式用 Schema。

9.4 Evaluator-Optimizer

Evaluator-Optimizer 是一种通用工作流：生成器产生产物，评价器根据标准给出反馈，循环直到达标或预算耗尽。Anthropic 将其列为常见有效 Agent 工作流之一 [20]。

9.4.1 何时适用

只有当评价标准可明确表达、迭代确实能改善结果时才适用。翻译润色、研究报告、代码审查和策略文档通常合适；开放创意或评价器本身不可靠时，循环可能只增加成本。

9.5 Generator-Verifier-Repair 三段式

生产系统常采用三个独立组件：Generator 生成候选；Verifier 运行硬验证和软评价；Repair 根据结构化错误修复。Verifier 不应只返回“好/不好”，而要给出错误位置、类别、严重度、可修复性和证据。

9.6 确定性验证优先

任务类型	首选验证器	模型 Critic 的辅助作用
代码修改	编译、单测、集成测试、静态分析	解释失败、寻找遗漏测试、审查语义风险
结构化输出	JSON Schema、类型系统、业务规则	判断字段语义是否充分
事实报告	多源检索、引用存在性、时间戳检查	识别论证缺口与矛盾
SQL/数据分析	只读执行、约束检查、对账查询	判断指标定义与业务解释

任务类型	首选验证器	模型 Critic 的辅助作用
高风险动作	权限策略、审批、沙箱、事务控制	给出风险摘要，不替代授权

关键结论

“让另一个模型检查”只是软验证。只要问题能被程序、规则、类型、测试或权限系统确定地验证，就不应把最终判断交给模型。

记忆中心 Agent: RAG、MemGPT、Generative Agents 与技能库

10.1 上下文、状态与记忆的区别

上下文是当前模型调用可见的信息；状态是系统执行的事实；记忆是跨步骤或跨会话保留、可再次检索的信息。把全部聊天记录持续拼接不等于记忆架构，只是无控制的上下文膨胀。

10.2 检索增强 Agent

RAG Agent 在行动前主动检索外部知识，并根据结果决定继续检索、调用工具或回答。其模块包括查询重写、检索器、重排器、证据存储、引用映射和答案合成。

10.2.1 Agentic RAG 与单次 RAG

单次 RAG 固定执行一次检索；Agentic RAG 可根据缺口多轮检索、切换数据源、验证冲突并停止。后者更灵活，但必须控制查询漂移和重复检索。

10.2.2 设计原则

检索结果必须保留来源、时间、权限和片段边界。记忆检索与事实检索应区分：前者可能来自 Agent 自己写入的经验，可信度通常低于权威数据源。

10.3 MemGPT/Letta：虚拟上下文管理

MemGPT 借鉴操作系统的分层存储思想，把有限上下文视为“主存”，把长期信息放在外部存储，并由 Agent 触发数据换入换出和控制中断 [11]。

10.3.1 分层结构

常见层次包括固定系统指令、核心记忆、当前消息、近期摘要、归档记忆和外部文档。Agent 需要决定哪些信息常驻、哪些压缩、哪些按需检索。

10.3.2 优势与风险

优势是支持长会话和跨会话连续性；风险是模型错误写入、过时信息长期驻留、用户与任务记忆混淆。必须提供来源、修改历史、删除机制和写入策略。

10.4 Generative Agents: 记忆流、反思与计划

Generative Agents 使用自然语言记忆流记录经历，通过相关性、重要性和新近性检索，并周期性生成高层反思，再基于记忆与反思制定行为计划 [12]。

10.4.1 架构启示

记忆不是平铺日志，而是经历、摘要和抽象知识的层次结构。反思层可以把多个低层事件归纳为稳定认识，但归纳必须可追溯回原始事件。

10.4.2 适用场景

适合模拟角色、长期个人助理、客户关系和持续项目协作。对于严格事实系统，需要额外的权威数据源，不能把 Agent 的主观总结当作事实数据库。

10.5 Voyager: 自动课程与可执行技能库

Voyager 在 Minecraft 中组合自动课程、可执行代码技能库和基于环境反馈的迭代改进，实现开放式终身学习 [13]。

10.5.1 技能库机制

成功策略被保存为可检索、可组合的代码技能，而不是只保存对话。新任务先检索相关技能，再调用或改写。程序性记忆可减少重复规划，并让复杂行为逐渐组合。

10.5.2 对通用 Agent 的启示

企业 Agent 也可把稳定成功流程沉淀为 Skill：输入条件、步骤、工具、验证、失败处理和权限要求。Skill 必须版本化、测试和审核；不能把偶然成功轨迹直接升级为全局技能。

10.6 记忆写入策略

推荐采用“候选写入 → 规则过滤 → 去重与冲突检测 → 人工或高置信审批 → 版本化存储”的流程。每条记忆包含来源、作用域、有效期、置信度和敏感级别。读取时应根据任务、用户、权限和时间过滤。

10.7 遗忘与压缩

记忆系统必须支持 TTL、显式删除、冲突合并、摘要重建和来源失效。压缩不只是缩短文本，还应保留决策、未解决问题、关键证据和后续动作。对长任务，应把可恢复状态写入结构化工件，而不是依赖模型记住全部轨迹。

图 workflow、状态机、事件驱动与耐久执行

11.1 顺序 workflow

最简单的 Agent workflow 是固定顺序：分类 → 检索 → 生成 → 验证 → 交付。只有个别节点调用模型。它适合过程稳定、异常少、需要高可预测性的场景，也是新系统的优先起点。

11.2 条件图与循环图

图架构把步骤建模为节点，把条件分支和循环建模为边，状态对象在节点间传递。LangGraph 强调把确定性步骤与模型驱动步骤混合，并提供持久化、人类中断和耐久执行能力 [74, 75]。

11.2.1 状态设计

状态应包含任务输入、规范化目标、工件引用、节点状态、错误、预算、审批记录和版本。大文件放对象存储，图状态只保存引用和摘要。节点必须声明读写字段，避免并行更新互相覆盖。

11.2.2 路由设计

边可以由规则、模型分类器或验证结果决定。优先用规则处理硬条件，例如权限、金额、测试结果；仅在语义边界模糊时使用模型路由。

11.3 DAG 与数据流架构

DAG 适合依赖已知且可并行的任务。调度器根据依赖计数启动 Ready 节点，节点产物可缓存和重用。研究报告、数据管道、批量代码审查和多源比较都适合 DAG。

DAG 不适合需要频繁回环和动态新增任务的开放 Agent，此时可使用动态图或“DAG 外层 + 局部 Agent Loop”。

11.4 事件驱动 Agent

事件驱动架构把用户消息、工具结果、文件变化、Webhook、定时器和审批都表示为事件。组件订阅事件并产生新事件，控制权不依赖一个持续占用进程的模型调用。

11.4.1 优势

组件解耦、易于异步、适合长任务和多入口；每个事件可持久化，便于重放和审计。OpenHands 的公开论文采用事件流连接界面、Agent 与环境 [35]。

11.4.2 难点

需要处理至少一次投递、重复事件、乱序、幂等和死信队列。事件 Schema 必须版本化，且敏感数据不能无限保留。

11.5 耐久执行

耐久执行意味着进程重启、超时或人工暂停后，任务能从最后确认的检查点继续，而不是从头重放所有工具。检查点应在具有副作用的动作前后保存，并记录幂等键、外部事务 ID 和补偿动作。

11.6 Human-in-the-Loop

人工审批不应只是弹窗，而是可持久化的暂停状态。系统保存待审批动作、参数、影响范围、证据和替代方案；审批后使用原始版本继续，若上下文已变化则重新校验。LangGraph 的 Interrupt 与 Checkpoint 就体现了这种耐久暂停模式 [76]。

黑板与共享工作空间架构

12.1 经典黑板架构

黑板架构由共享黑板、多个知识源和控制器组成。知识源读取黑板状态，提出局部贡献；控制器选择下一步激活哪个知识源。该架构适合多个异质专家逐步构建同一解答 [19]。

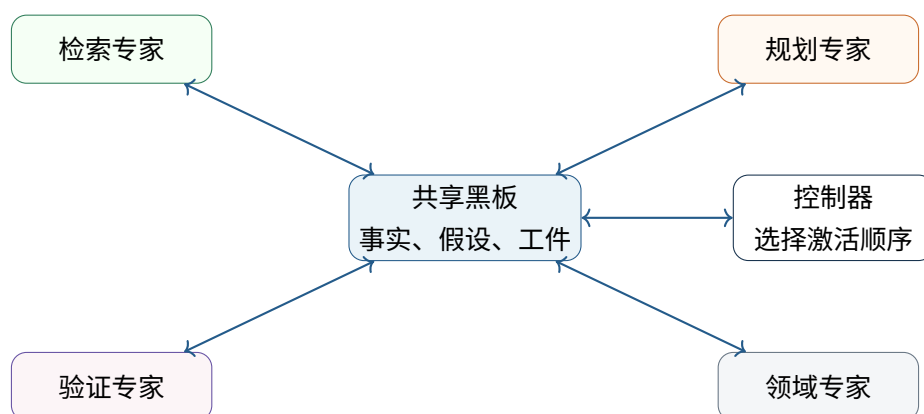


图 12.1: 黑板架构：专家通过共享工件协作，控制器管理激活顺序。

12.2 黑板中的信息类型

黑板不应只是共享聊天记录。可分层存储原始证据、结构化事实、候选假设、任务列表、工件、冲突、风险和最终决策。每条写入必须带作者、来源、版本和置信度。

12.3 控制策略

控制器可按优先级、信息增益、依赖、成本或风险选择知识源。简单系统可使用规则；复杂系统可由模型推荐，但最终调度仍应遵守资源和权限约束。若所有专家都能随意写入所有区域，黑板会迅速污染。

12.4 适用场景

适合代码审计、医疗多学科讨论、复杂研究、事故分析、情报融合和大型项目设计。多个专家贡献不同证据，最终决策建立在共享工件上，而非互相转述长对话。

12.5 与群聊式多 Agent 的差异

群聊把消息作为主要协作介质；黑板把共享状态和工件作为主要介质。群聊容易理解，但上下文冗余大、结论难追踪；黑板更适合并行和审计，但需要定义 Schema、冲突解决与写权限。

单智能体混合架构与选型

13.1 常见混合方式

生产 Agent 通常同时包含多种模式：外层 FSM 管理生命周期, HTN 或 Plan-and-Execute 产生任务图, 局部节点使用 ReAct/CodeAct, 结果经 Verifier 验证, 长期信息进入记忆系统, 整个过程运行在耐久工作流上。

13.2 六个选型维度

1. **任务可预测性**：步骤是否能提前枚举；
2. **环境可逆性**：错误动作能否回滚；
3. **验证强度**：是否存在确定性成功信号；
4. **时延与成本**：能否承受多轮模型和搜索；
5. **风险与合规**：是否需要审批、审计和固定流程；
6. **持续时间**：是秒级交互还是跨天任务。

13.3 选型矩阵

任务特征	首选架构	可叠加模块	不宜优先采用
流程固定、风险高	FSM/行为树/HTN	审批、验证器、耐久执行	自由 ReAct
开放探索、反馈及时	ReAct/CodeAct	反思、候选池、工具检索	固定长 DAG
依赖清楚、可并行	显式计划 / DAG / LLMCompiler	缓存、重试、成本调度	全串行工具循环
候选多、可回滚、价值高	Beam/ToT/LATS	强验证器、沙箱快照	单轨迹贪心
跨会话持续协作	BDI/记忆中心/事件驱动	定时器、身份、TTL	仅拼接聊天历史

任务特征	首选架构	可叠加模块	不宜优先采用
多个异质专家共享证据	黑板	控制器、冲突检测	无结构群聊

关键结论

架构越“智能”不代表越好。能用单次模型调用解决的任务，不需要 ReAct；能用固定工作流解决的任务，不需要自由规划；能用单 Agent 完成的任务，不需要多 Agent。

第二部分

多智能体架构与协作拓扑

多智能体系统的基本问题

14.1 为什么拆成多个 Agent

多 Agent 的合理动机主要有四类：不同子任务需要不同工具或权限；上下文需要隔离；子任务可并行；需要独立的生成与审查角色。仅仅为了“模拟一个团队”而复制多个相同模型，通常只会增加成本和协调误差。

14.2 五种拓扑维度

多 Agent 架构可从控制中心化程度、消息可见范围、共享状态方式、任务分配机制和终止决策者五个维度描述。中心化 Supervisor 易治理；去中心化 Handoff 更灵活；共享群聊简单但上下文昂贵；黑板和工件库更适合规模化。

14.3 Agent、工具与子工作流的边界

只有具备独立目标、状态、策略和行动能力的组件才需要被建模为 Agent。一个只执行固定查询的组件更适合作为工具；一组确定性步骤更适合作为子工作流。过度 Agent 化会让控制流、错误语义和测试变得模糊。

14.4 通信契约

Agent 间消息应包含任务 ID、发送者、接收者、意图、输入工件、预期输出、截止时间、权限和追踪上下文。自然语言可用于语义内容，但任务状态和工件引用应结构化。所有跨 Agent 委派都需要明确“谁负责最终验收”。

路由、Handoff 与 Swarm

15.1 中心化 Router-Specialist

Router 读取用户请求并选择领域 Specialist。Specialist 具有独立提示、工具、记忆和权限，完成后把结构化结果返回 Router 或直接交付。

15.1.1 路由策略

可使用规则、轻量分类器、向量相似度或 LLM。高风险路由应同时检查身份和权限，不能仅根据语义选择。路由器还要处理多标签任务：拆分为多个 Specialist，或交给能协调子任务的通用 Agent。

15.1.2 优缺点

优势是模块边界清楚、成本可控、容易做领域评测；不足是路由错误会直接导致失败，跨领域任务需要额外协调。适合企业客服、IT 助手、医疗科室分流和多产品入口。

15.2 Handoff 网络

Handoff 由当前 Agent 主动把控制权移交给另一个 Agent。与 Router 只在开头决策不同，Handoff 可在执行过程中动态发生。OpenAI Agents SDK 将 Handoff 作为独立编排原语 [71]。

15.2.1 共享与隔离

共享上下文便于无缝切换，但会泄露无关信息并消耗 Token；隔离上下文更安全，但需要交接摘要和工件。推荐传递最小任务包，而不是完整对话。

15.2.2 防止乒乓

必须记录 Handoff 链、最大次数和原因。接收方若拒绝任务，应返回结构化原因，而不是立即交回。可设置所有权字段，只有当前 Owner 有权继续委派。

15.3 去中心化 Swarm

Swarm 让各 Agent 根据本地判断选择下一个 Agent,而不依赖中央 Supervisor。AutoGen 的 Swarm 团队采用基于 Handoff 消息的局部委派 [30]。

15.3.1 适用场景

适合能力边界清楚、协作路径动态、中央规划困难的系统,例如复杂客服、跨部门流程和开放式专家网络。

15.3.2 风险

局部决策可能无法优化全局成本,容易形成环路、责任不清和上下文扩散。需要共享任务状态、全局预算、审计链和兜底协调器。

Supervisor-Workers 与 Orchestrator 架构

16.1 基本结构

Supervisor 负责理解目标、拆分任务、选择 Worker、监控进度、处理失败和汇总结果；Worker 专注于搜索、代码、浏览器、数据或其他能力。它是最常见的多 Agent 拓扑。

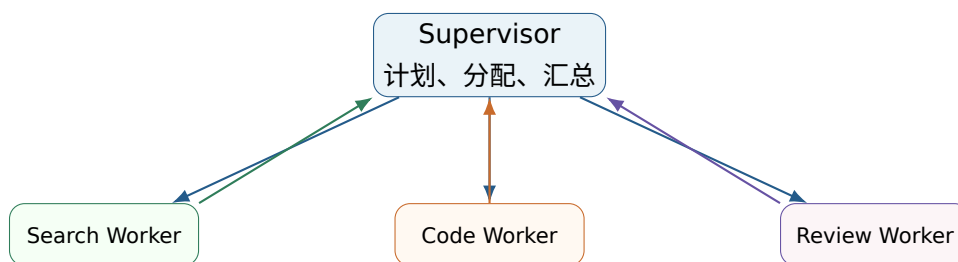


图 16.1: Supervisor-Workers: 中央协调、专门执行、结构化回传。

16.2 静态与动态分解

静态 Supervisor 从预定义角色中选择；动态 Supervisor 根据任务创建临时 Worker。动态方式灵活，但更难控制工具权限和提示质量。生产系统可允许动态任务，不允许动态权限：Worker 角色可以临时实例化，但能力集合必须来自批准模板。

16.3 Map-Reduce 型并行

任务先被拆成互相独立的 Map 子任务并行执行，再由 Reduce Agent 去重、比较、合并和验证。适合文档集合分析、代码库搜索、多源研究和候选方案评估。

Reduce 阶段不能简单拼接。它需要处理证据冲突、覆盖率、重复和置信度，并保留每个结论来自哪个 Worker。

16.4 Anthropic Research 的 Orchestrator-Worker

Anthropic 公开的多 Agent 研究系统使用 Lead Agent 协调多个并行子 Agent，对不同方向进行搜索，再由 Lead 汇总 [21]。这一模式特别适合宽度优先的信息搜集，因为并行 Worker 可以覆盖不同子主题。

16.5 Magentic-One

Magentic-One 使用 Orchestrator 规划、跟踪进度并在错误后重规划，同时协调 Web-Surfer、FileSurfer、Coder 等专门 Agent[28, 29]。其关键不是角色名称，而是 Orchestrator 维护任务账本和进度事实。

16.6 Supervisor 的失败模式

Supervisor 可能分解错误、无法判断 Worker 结果、重复委派或过早终止。缓解方式包括：任务对象结构化、Worker 返回证据与自检、独立 Verifier、共享工件、最大并行度和任务覆盖检查。

群聊、角色扮演与动态发言者选择

17.1 多 Agent Conversation

AutoGen 将可对话 Agent 作为核心抽象，Agent 可组合模型、工具和人类输入，并通过可编程会话模式协作 [24]。群聊架构的主要变量是下一位发言者如何选择、消息对谁可见以及何时终止。

17.2 Round-Robin

轮询让 Agent 按固定顺序发言，简单且可预测。适合固定审查会议或短链路协作。问题是即使某角色没有新增价值也会发言，产生冗余和“礼貌性赞同”。

17.3 Selector Group Chat

Selector 使用模型根据共享上下文动态选择下一位发言者 [31]。它能减少无关轮次，但 Selector 自身可能偏向某角色或陷入循环。应使用角色能力描述、禁止连续发言、最大轮数和低置信回退。

17.4 CAMEL 角色扮演

CAMEL 通过角色设定和 inception prompting 引导两个或多个 Agent 自主对话完成任务 [25]。角色可提供目标、知识边界和交流风格，使任务分工更明确。

17.4.1 局限

角色扮演容易产生“表演式协作”：Agent 重复复述角色职责，却没有独立工具、证据或状态。角色只有在能力、权限、上下文或评价标准真正不同的情况下才有架构意义。

17.5 对话去幻觉与共识

群聊系统可要求每轮产出“新证据、反例、未解决问题或决策”，禁止无信息增量发言。共识不等于正确，应允许保留少数意见，并由外部验证器决定可检验事实。

SOP、流水线与组织模拟：MetaGPT 和 ChatDev

18.1 SOP 驱动的多 Agent

SOP 架构把人类组织流程编码为角色、阶段、输入产物、输出产物和质量门。它比自由群聊更接近软件工程流水线。

18.2 MetaGPT

MetaGPT 将标准操作流程编码进多 Agent 协作，并以产品经理、架构师、工程师等角色生产结构化中间产物 [26]。其装配线思想强调每个角色消费前序工件并生成后序工件。

18.2.1 架构要点

真正的核心是工件契约，而不是角色名称。需求文档、接口设计、任务列表、代码和测试应有 Schema、版本和验收规则。若只让多个 Agent 以自然语言聊天，无法获得 SOP 的可靠性。

18.3 ChatDev

ChatDev 使用由设计、编码、测试和文档等阶段组成的 Chat Chain，并通过角色间对话完成软件开发 [27]。其“communicative dehallucination”通过问答细化和交叉检查降低信息不一致。

18.3.1 优缺点

优势是阶段清晰、角色知识可配置；不足是早期错误可能沿流水线传播，后续 Agent 容易默认前序产物正确。应为每个 Gate 增加独立验证、置信度和返工路径。

18.4 流水线架构的通用化

SOP 模式适用于合同审查、营销内容、数据报告、安全响应和科研流程。每阶段应定义：责任角色、输入、输出、硬校验、软评价、最大返工次数和升级条件。

Debate、Committee、Ensemble 与 Judge

19.1 多 Agent Debate

Debate 让多个 Agent 独立提出答案，交换论证或批评，再由参与者或 Judge 选择结果。它可暴露单一路径忽略的反例，适合推理、决策和审查。

19.1.1 独立性要求

若所有 Agent 共享相同上下文、提示和初始答案，它们的错误高度相关。应通过不同检索源、不同角色、不同模型或先独立生成后交流来提高多样性。

19.1.2 终止与裁决

Debate 需要固定轮数、论点结构和 Judge Rubric。Judge 应引用证据，而不是根据表达流畅度选择。高风险任务仍需外部事实或规则验证。

19.2 Committee 与投票

Committee 并行生成多个候选，通过多数投票、加权投票或置信度校准选择。适合分类、短答案和可独立采样的任务。开放文本难以直接投票，可先抽取结构化主张再比较。

19.3 Mixture-of-Agents

Mixture-of-Agents 把多个模型的输出作为后续聚合模型的输入，利用模型间互补性。架构重点是层数、候选多样性、聚合器和成本控制。它更像推理时集成，而非持续行动 Agent。

19.4 Generator-Critic Committee

可以设置多个专门 Critic：事实、逻辑、安全、合规、风格和成本。每个 Critic 只检查一个维度并输出结构化问题，最终仲裁器根据严重度处理。这样比让一个“万能 Reviewer”更容易评测。

19.5 何时不要使用

没有可靠裁决标准、模型错误高度相关、任务成本低或时间敏感时，多 Agent Debate 通常不划算。增加人数不会自动产生真实独立性。

黑板多 Agent、Agent-as-Tool 与市场式协作

20.1 多 Agent 黑板

多个 Agent 共享一个结构化工作区，分别提交证据、假设、任务和工件。协调器根据黑板缺口分配下一任务。与 Supervisor 直接发消息相比，黑板降低了重复转述，并允许异步 Worker 加入和离开。

20.2 Agent-as-Tool

上层 Agent 可以把下层 Agent 当作高层工具调用。下层 Agent 有自己的循环和工具，但只向上层返回受约束结果。此模式适合把 Coding Agent、Research Agent 或 SQL Agent 封装成能力服务。

20.2.1 与 Handoff 的区别

Agent-as-Tool 不转移用户对话的控制权，上层仍负责最终答案；Handoff 则把控制权交给另一个 Agent。前者适合任务委派，后者适合服务所有权切换。

20.3 Contract Net 与竞标

Contract Net 是经典多智能体任务分配模式：管理者发布任务，候选 Agent 报价，管理者选择承包者，承包者执行并回报。报价可包含成本、预计时间、置信度和资源需求。

20.3.1 适用场景

适合异构算力、多个模型供应商、跨组织代理和动态资源调度。必须防止虚假报价、重复承包和预算失控，通常需要信誉、限额和结果验收。

20.4 市场式与效用优化

更复杂系统可为任务、工具和计算资源定价，让 **Agent** 在预算内选择方案。但 **LLM** 并不天然具备稳定经济理性，市场机制只能作为调度启发式，不能替代安全策略和业务规则。

多 Agent 失败模式与选型准则

21.1 协调开销

每增加一个 Agent，就增加消息、上下文、等待、错误边界和终止条件。若子任务不可并行或没有专业差异，多 Agent 往往比强单 Agent 更慢、更贵。

21.2 幻觉传播

早期 Agent 生成的错误工件会被后续角色当作事实。解决方式是标注来源与置信度、每阶段 Gate 验证、允许退回和保留不确定性，而不是要求后续 Agent “相信团队成员”。

21.3 共享上下文污染

群聊中任何 Agent 的长输出都会进入所有人的上下文，造成注意力稀释和敏感信息扩散。应使用最小交接包、工件引用、按需读取和角色级访问控制。

21.4 写冲突与并行副作用

并行 Coding Agent 修改同一文件、多个业务 Agent 操作同一账户都可能冲突。需要 Work-tree、乐观锁、事务、幂等键、资源租约和冲突合并。

21.5 责任与身份

每个 Agent 必须有独立身份、权限和审计记录。组织型 Agent 不应冒充某个人；OpenClaw 的 Delegate 架构也强调以独立身份代表组织成员行动 [67]。

21.6 使用多 Agent 的判断问题

1. 子任务是否真的独立或需要不同能力？
2. 是否能定义清晰的输入输出契约？

3. 是否有可靠的汇总和验收机制?
4. 并行收益是否超过协调成本?
5. 敏感上下文是否能隔离?
6. 出错时能否确定责任与恢复点?

第三部分

代表性产品与功能型 Agent 架构

Coding Agent 的通用架构

22.1 Coding Agent 的任务闭环

Coding Agent 面向仓库级软件工程任务，必须完成理解、定位、修改、执行和验证，而不仅仅是生成代码片段。典型闭环为：任务契约 → 仓库勘察 → 复现/定位 → 计划 → 编辑 → 测试 → Diff 审查 → 交付。

22.2 任务契约

任务输入应包含目标、禁止变化、验收标准、目标分支、可用环境和风险等级。Issue 描述不一定可靠，Agent 需要区分用户期望、现有行为和测试事实。

22.3 仓库理解层

常见能力包括目录树、文本搜索、AST/符号索引、LSP、调用图、Git 历史、依赖图、构建配置和测试映射。检索顺序应从模块到符号再到局部代码，避免一次读取整个仓库。

22.4 编辑层

编辑接口包括整文件重写、搜索替换、Patch、AST 重构和 IDE 操作。Patch 便于审查与回滚；AST 更稳健但覆盖语言有限；整文件重写适合小文件。最终都应统一呈现为 Diff。

22.5 验证层

验证顺序通常为复现脚本、目标测试、相关测试、类型/Lint、全量测试和语义审查。测试必须区分修改前基线与修改后结果。Agent 不得把“命令未运行”表述为“测试通过”。

22.6 环境与权限

本地工作区适合交互；Git Worktree 适合并行隔离；容器/VM 适合依赖和进程隔离；云沙箱适合长任务。网络、凭据和写目录需要最小权限，构建脚本视为不受信任代码。

22.7 长任务架构

长程编码任务需要任务账本、进度文件、检查点、可恢复环境和增量提交。Anthropic 公开的长任务 Harness 使用初始化 Agent 建立环境，再由后续 Coding Agent 分会话增量推进并留下清晰工件 [22]。

22.8 子 Agent 的合理使用

可并行搜索调用链、阅读测试、审查安全和性能，但写操作必须隔离。主 Agent 应为每个子任务定义交付物和证据，并负责最终合并。

Claude Code 架构详解

23.1 公开可确认的核心循环

Claude Code 的公开文档将工作过程描述为收集上下文、采取行动和验证结果的循环 [40]。它能读取代码库、编辑文件、运行命令并在终端、IDE 等界面中工作。

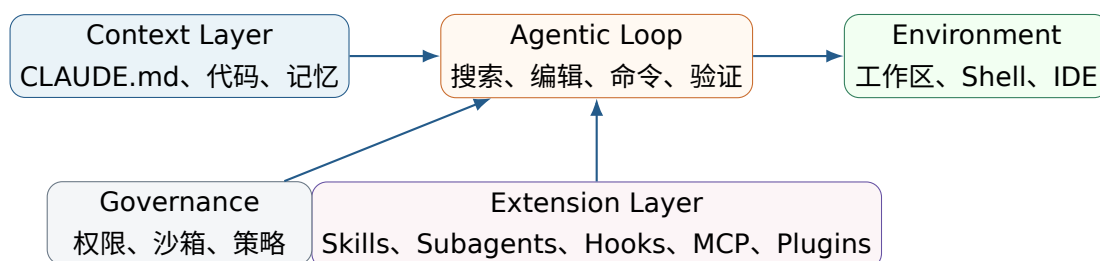


图 23.1: Claude Code 的教学抽象：上下文、循环、环境、扩展和治理。

23.2 上下文层

CLAUDE.md 用于项目级约定，.claude/ 与用户级目录可承载设置、规则、技能、子 Agent、Hook 和自动记忆 [45, 46]。稳定且高频的规则应放在项目说明；低频流程应做成 Skill；大量项目知识应按需检索。

23.3 工具循环

模型根据任务搜索与读取文件、运行命令、编辑并检查结果。真正的架构价值来自工具反馈闭环：编译器、测试和 Git Diff 给出环境真相，模型据此调整，而不是一次性生成最终补丁。

23.4 Hooks：生命周期上的确定性控制

Hooks 可在特定生命周期事件上运行 Shell、HTTP、MCP 工具、LLM Prompt 或 Sub-agent [43]。它们适合格式化、阻止危险命令、补充日志、强制测试和审计。

23.4.1 Hook 与 Prompt 的边界

“尽量运行格式化”适合提示；“每次编辑 Python 文件后必须运行格式化器”适合 Hook。Prompt 影响概率，Hook 固定触发点。Hook 本身也需要超时、错误处理和最小权限。

23.5 Subagents

Subagent 在独立上下文中处理专门任务，可收窄工具并配置独立记忆 [42, 45]。适合大量搜索、日志分析和独立审查。主 Agent 应传递明确任务包，并验证返回内容，不能把 Subagent 当作天然可信专家。

23.6 Skills、Plugins 与 MCP

Skill 打包领域说明、资源和流程；Plugin 可组合 Skill、Hook、Subagent 和 MCP；MCP 连接外部工具与数据 [44, 41]。这形成“程序性知识、生命周期控制、专门执行、外部能力”的分层扩展体系。

23.7 记忆与压缩

Claude Code 公开文档区分主会话和 Subagent 的自动记忆，主题文件按需读取 [45]。长会话还需要上下文压缩。架构上应将持久事实放文件或状态存储，把对话摘要视为缓存，而非唯一事实来源。

23.8 权限与沙箱

安全架构包含只读默认、敏感动作审批和 Shell 的文件/网络隔离 [47]。需要区分权限决策与沙箱边界：即使模型获准执行命令，操作系统仍应限制可访问资源。

23.9 长任务 Harness

长任务不应依赖一个无限上下文会话。可用初始化阶段建立环境、任务列表和进度工件，后续会话每次完成可验证增量并更新工件 [22]。这样可以跨上下文窗口恢复。

23.10 适用场景与局限

Claude Code 适合交互式本地开发、仓库理解、调试和可扩展终端工作流。主要风险是长轨迹偏航、项目隐含约束、权限过宽和并行写冲突。解决方式是项目说明、Hooks、测试、Worktree、最小权限和结构化进度，而不是无限增加提示词。

Codex 架构详解

24.1 多表面与多环境

Codex 提供 CLI、IDE、桌面/ChatGPT 界面和云任务等使用表面。公开文档强调本地仓库中的探索、编辑和运行，以及 Worktree/Cloud 等隔离执行方式 [48, 49, 57]。

24.2 Local、Worktree 与 Cloud

Local 直接使用当前工作区，反馈快但可能干扰用户；Worktree 为并行任务提供独立 Git 工作树；Cloud 在远程隔离环境中运行，适合长任务、并行和团队协作。架构选择取决于数据边界、依赖、时长和并发，而不是模型本身。

24.3 AGENTS.md 层次化指令

Codex 在工作前读取 AGENTS.md，并支持全局与目录级覆盖 [50]。大型 Monorepo 可在根目录放全局规则，在子目录放服务特有构建、测试和架构约束。

24.4 Skills 与定制层

Codex Skill 打包说明、资源和可选脚本，使重复 workflow 更稳定 [51]。定制层还包括 MCP、项目说明和自定义 Subagent。稳定流程应沉淀为 Skill，临时任务留在用户指令中。

24.5 Subagents 与并行线程

Codex 可并行启动专门 Subagent 并汇总结果 [53]。适合代码库探索、方案比较、实现与审查分离。不同写任务应使用独立 Worktree，避免共享文件冲突。

24.6 沙箱与审批

Codex 将 Approval Mode 与 Sandbox Mode 分开：前者决定何时请求授权，后者限制进程可以访问什么资源 [54, 55]。安全底线应由操作系统级边界实现，而不是由模型或 Reviewer 的判断实现。

24.7 自动化与长程任务

Automations 可以按计划触发工作；长任务实践强调 AGENTS.md、Skills、Subagents、Worktrees 和可审查输出 [56, 57]。自动化任务必须幂等，避免周期性重复创建相同改动。

24.8 多 Agent 控制台思路

Codex App 公开描述了多个 Agent 在独立线程中并行工作、审查 Diff 和管理长任务的模式 [58]。这使产品从“单会话 Coding Agent”发展为“软件工程任务控制台”。

24.9 与 Claude Code 的架构比较

两者都包含工具循环、项目指令、扩展、Subagent、隔离和审批。Claude Code 更突出 Hooks、项目记忆和终端扩展生态；Codex 更突出 Local/Worktree/Cloud 统一任务环境、并行线程、Skills 和 Automations。选择应基于工作流与治理，而非只比较单次生成能力。

开源 Coding Agent 架构

25.1 SWE-agent: ACI 优先

SWE-agent 的核心观点是 Agent-Computer Interface 会显著影响软件工程 Agent 能力 [34]。它通过专门命令和反馈格式，让模型更容易搜索、浏览和编辑仓库，而不是直接暴露任意终端输出。

25.1.1 设计启示

工具名称、输出长度、错误信息和编辑方式都是架构的一部分。一个适合人的 Shell 不一定适合模型；高层、可恢复、结构化的工具往往比完全开放 Bash 更可靠。

25.2 OpenHands: 事件流与 CodeAct

OpenHands 公开论文提出事件流架构连接界面、Agent 和环境，并提供包含 Shell、浏览器和 Python 的沙箱运行时 [35]。默认 Agent 使用 CodeAct 风格，通过代码与环境交互。

25.2.1 平台化意义

事件流使不同 UI、Agent 和 Runtime 可替换，适合实验与部署。后续 Software Agent SDK 强调可组合 Agent、沙箱执行、生命周期控制和本地到远程运行 [36]。

25.3 Aider: Repository Map 与 Architect/Editor 分离

Aider 使用 Repo Map 向模型提供文件与关键符号的压缩视图 [37]。这种图/符号驱动上下文比盲目向量检索更贴合代码结构。

Architect/Editor 模式把高层方案与具体编辑分离：Architect 负责推理和说明修改，Editor 负责按编辑格式落地。它体现了“规划模型与执行模型可分离”的思想。

25.4 Agentless: 确定性流水线基线

Agentless 使用定位、修复和补丁验证的简单阶段，不让模型自由决定下一动作 [38]。它提醒设计者：某些软件问题用高质量定位与候选生成流水线即可，不一定需要长 ReAct 轨迹。

25.5 MASAI: 模块化软件工程 Agent

MASAI 将软件问题拆给具有不同目标和策略的子 Agent，以减少无关长轨迹，并从不同仓库来源收集信息 [39]。它代表“专门模块 + 明确交付物”的多 Agent Coding 架构。

25.6 五种系统的对比

系统/方法	核心架构重点	适合学习的设计	主要限制
SWE-agent	ACI 与工具反馈	为模型设计命令、错误和编辑接口	仍依赖循环质量
OpenHands	事件流、CodeAct、沙箱	平台解耦与环境运行时	系统复杂度高
Aider	Repo Map、Git、编辑格式	Token 高效仓库上下文	更偏交互式协作
Agentless	定位-修复-验证流水线	先建立简单强基线	开放任务适应性有限
MASAI	专门子 Agent 模块	按策略与信息源分工	协调和汇总成本

表 25.1: 代表性开源 Coding Agent 架构重点。

OpenClaw：网关、会话、记忆与长期运行架构

26.1 定位

OpenClaw 更接近自托管的多渠道 Agent 网关与运行时，而非单一 Coding Agent。Gateway 将消息渠道、模型、会话、工具、技能、记忆、子 Agent 与自动化连接起来 [59]。

26.2 Gateway 控制平面

Gateway 是常驻进程，处理 Provider/渠道连接、RPC、事件、会话和定时任务。它把“始终在线的接入与调度”同“按消息执行的模型循环”分离 [60]。

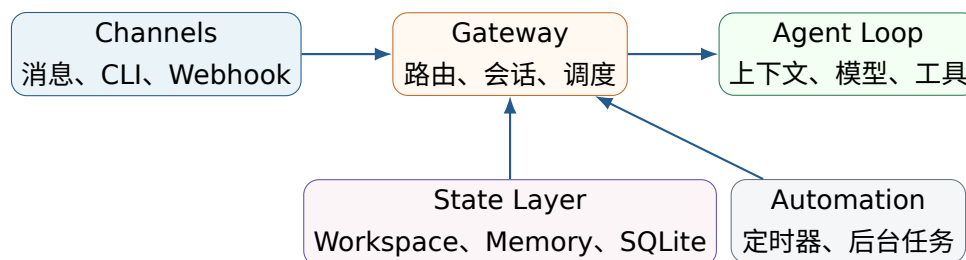


图 26.1: OpenClaw 的长期运行分层。

26.3 每会话串行 Agent Loop

公开文档将 Agent Loop 描述为每会话串行执行的消息处理过程，包括 intake、上下文组装、模型推理、工具执行、流式输出和持久化 [61]。每会话串行可避免同一会话的状态竞争，同时允许不同会话并行。

26.4 Workspace 与 Memory

每个 Agent 可有独立 Workspace、状态目录和会话历史；记忆文件区分用户稳定偏好、长期事实等内容 [62, 63]。明文文件便于审查和编辑，但必须保护敏感信息并提供删除机制。

26.5 Skills 与 Subagents

Skill 提供程序性能力；Subagent 是从当前运行派生的后台会话，完成后向请求方回传结果 [64, 65]。适合耗时搜索和并行子任务，但应限制后台任务数量和资源。

26.6 多 Agent 路由

同一 Gateway 可以运行多个隔离 Agent，每个 Agent 有独立 Workspace、状态和渠道绑定 [66]。这种架构适合不同身份、部门或用途的 Delegate，并避免把所有个人和组织上下文混在一个 Agent 中。

26.7 Automations

OpenClaw 内置持久化调度器，可在指定时间唤醒 Agent 并把结果发送到渠道或 Webhook [68]。长期自动化必须处理幂等、错过执行、时区、重复通知和任务过期。

26.8 Runtime 与 Harness

公开的运行时架构说明了内置 Runtime、Harness 注册、Provider 路由和插件边界 [69]。这使 OpenClaw 可以在统一 Gateway 下选择不同 Agent Runtime，而不把渠道层与特定模型实现耦合。

26.9 安全边界

需要分别控制渠道身份、Agent 身份、工具权限、Workspace、沙箱和网络。长期在线 Agent 面临更大的 Prompt Injection、凭据泄露和自动化误操作风险，因此默认权限应低于交互式临时 Agent。

26.10 适用与不适用

OpenClaw 适合自托管个人助理、多渠道组织 Delegate、持续自动化和跨会话记忆。若任务只是一次性代码修复或固定 API 流程，使用专门 Coding Agent 或普通 workflow 更简单。

研究与深度检索 Agent

27.1 典型分层

研究 Agent 通常由问题分解、查询规划、并行搜索、文档阅读、证据抽取、冲突分析、综合写作和引用验证组成。核心工件是 Evidence Ledger，而不是长对话。

27.2 问题分解

Lead Agent 将问题拆为互补子问题，标记时间范围、地域、定义和需要的来源类型。拆分应覆盖不同假设，而不是把同一查询改写多次。

27.3 并行搜索 Worker

不同 Worker 针对子问题、来源域或观点搜索。Worker 返回结构化证据：主张、来源、发布日期、摘录位置、可信度和未解决问题。Supervisor 根据覆盖缺口继续派发任务。

27.4 阅读与证据抽取

阅读 Agent 不直接写结论，而是抽取可引用主张和上下文。表格、图和 PDF 需要专门解析。所有二手结论应追溯到原始来源，避免引用链漂移。

27.5 综合与冲突

Synthesizer 对证据聚类，区分一致事实、来源分歧和推断。冲突不能用“多数来源”简单消除，应比较权威性、时间和方法。

27.6 引用验证

Citation Verifier 检查每个关键事实是否被引用支持、引用是否真正包含该信息、时间是否匹配。模型生成的引用文本必须由检索系统校验 URL/文档 ID。

27.7 架构选择

窄问题可用单 Agent ReAct；宽度大且子问题独立时用 Orchestrator-Workers；依赖明确时用 DAG；高要求报告可增加 Critic 和 Citation Verifier。

浏览器与 Computer-use Agent

28.1 感知-决策-动作闭环

浏览器 Agent 读取页面状态，选择点击、输入、滚动、导航或下载动作，再观察页面变化。状态表示可以是 DOM/可访问性树、截图视觉、网络请求或多模态组合。

28.2 DOM/Accessibility Tree 架构

结构化元素提供标签、角色和层级，便于精确定位和低 Token 操作。缺点是复杂 Canvas、虚拟化列表和视觉布局信息可能缺失。

28.3 纯视觉 Computer-use

视觉 Agent 根据截图和坐标操作，通用性高，可覆盖无结构界面；但定位误差、分辨率变化和隐蔽状态会降低可靠性。通常需要截图差分、元素检测、动作后确认和坐标校准。

28.4 混合感知

生产系统优先用 DOM/语义选择器，失败时回退视觉；同时读取 URL、页面标题和网络状态。感知结果要有稳定元素 ID，避免因页面重排点击错误对象。

28.5 任务规划

可用行为树或 FSM 固定登录、导航、填写、确认等关键顺序，局部使用 ReAct 处理页面差异。购买、发送、删除等不可逆动作前必须暂停审批并展示摘要。

28.6 安全问题

网页内容是不受信任输入。页面中的 Prompt Injection 不得修改系统规则或诱导调用高权限工具。浏览器会话、Cookie、下载文件和剪贴板都应隔离；下载内容先扫描再进入其他工具。

数据分析与 SQL Agent

29.1 架构目标

数据 Agent 要把自然语言问题转换为指标定义、数据源选择、查询、验证、分析和解释。最大风险不是 SQL 语法错误，而是口径错误、越权查询和看似合理的错误结论。

29.2 语义层

使用经过治理的指标、维度、实体和关系定义，让 Agent 优先选择业务语义而不是直接猜表名。语义层可提供同义词、时间口径、权限和示例查询。

29.3 Planner-Generator-Executor-Verifier

Planner 分解问题并选择数据源；Generator 生成 SQL/代码；Executor 在只读沙箱运行；Verifier 检查语法、扫描量、行数、空值、重复、对账和业务约束；Narrator 最后解释结果。

29.4 查询修复

执行错误可自动修复，但结果不报错不代表正确。应生成对照查询、边界样本、总量对账和已知指标比较。高成本查询先 Explain 或限制扫描范围。

29.5 权限与隐私

数据库凭据不进入模型上下文；服务端按用户身份执行行列级权限。结果需脱敏和聚合，导出动作单独审批。Agent 不能通过多次小查询绕过隐私阈值。

29.6 多 Agent 设计

复杂分析可由 Metric Agent、SQL Agent、Statistics Agent 和 Critic 分工，但共享工件应是查询、数据快照和分析 Notebook，而不是仅用群聊传递结论。

安全与漏洞分析 Agent

30.1 分层流水线

安全 Agent 可分为攻击面建模、代码/配置检索、候选生成、可达性分析、动态验证、误报反思、修复和报告。不同阶段需要不同工具、权限和验证标准。

30.2 知识与模式层

漏洞模式应结构化为脆弱模式、触发条件、数据流、危险 Sink、必要前置条件、PoC 草图和排除规则。检索到模式后必须结合目标仓库语义验证，不能仅按相似代码判定。

30.3 静态与动态协同

Static Agent 使用 AST、调用图、数据流和规则定位候选；Dynamic Agent 在隔离环境复现；Verifier 判断真实可达性和影响。静态结果提供宽度，动态验证提供精度。

30.4 Harness

Harness 负责构建、依赖、服务启动、输入注入、日志收集和成功判据。Agent 的漏洞能力很大程度取决于 Harness 是否能稳定复现真实用户入口，而不是直接调用内部函数。

30.5 Memory 与反作弊

Episodic Memory 可记录误报原因和失败尝试；Semantic Memory 保存攻击面和信任边界。必须识别通过修改测试、绕过验证或利用沙箱缺陷获得奖励的 Reward Hacking。

30.6 权限和隔离

漏洞验证环境应与生产网络、凭据和用户数据隔离；出站网络按需开放；执行资源设限；PoC 和恶意样本按敏感工件管理。自动提交报告或利用链前需要人工审批。

30.7 适合的多 Agent 拓扑

可使用 Supervisor 协调 Pattern、Static、Dynamic、Exploitability 和 Patch Agent; 也可使用黑板共享证据。最终结论必须由可复现证据和明确判据支持, 而不是多 Agent 投票。

具身、游戏与机器人 Agent

31.1 层次控制

具身 Agent 通常使用高层目标规划、中层技能选择和低层控制三层。LLM 负责理解任务和选择技能，运动规划器/控制器负责连续动作，安全层拥有最高优先级。

31.2 感知与世界状态

感知模块将视觉、位置和传感器转换为对象、关系和可行动区域。世界模型保存地图、物体状态和不确定性。模型无法直接从一张截图可靠维护长期空间状态，因此需要外部定位和状态估计。

31.3 技能库

技能是可复用的时序行为，例如导航、抓取、开门或合成物品。Voyager 展示了可执行技能库和自动课程在开放环境中的价值 [13]。机器人技能还必须声明前置条件、资源、成功判据和安全约束。

31.4 行为树与 LLM 的结合

行为树负责安全回退、持续 Tick 和动作顺序；LLM 生成目标或选择子树。紧急停止、碰撞避免和力限制必须由低层确定性控制执行。

31.5 仿真与现实

高风险动作先在模拟器或数字孪生中验证。Sim-to-real 差异需要在线监控和保守阈值。任何能影响物理世界的 Agent 都应具备硬件急停和人工接管。

客服、个人助理与长期运行 Agent

32.1 客服 Agent

客服系统通常采用意图 Router、领域 Specialist、知识检索、业务工具、政策 Guardrail 和人工升级。会话状态包括用户身份、工单、已完成步骤和待确认动作。

32.1.1 交易型与解释型任务

解释政策可由检索生成；退款、改签和账户操作必须走固定工作流与权限校验。模型不能把自然语言同意当作高风险交易授权，必要时使用二次确认。

32.2 个人助理

个人助理需要 BDI 式目标管理、跨会话记忆、日历/邮件等工具、事件驱动唤醒和身份边界。用户偏好、临时任务和敏感事实必须分开存储。

32.3 长期运行

长期 Agent 通过定时器、Webhook 或条件事件启动。每次运行应是独立、可重试、幂等的任务；长期目标保存在任务数据库，模型只处理当前迭代。

32.4 主动性控制

主动 Agent 必须有通知预算、静默时段、去重和相关性阈值。没有新信息时不应打扰用户。自动发信、下单、删除和公开发布等动作需明确授权。

32.5 OpenClaw 式 Gateway 的价值

Gateway 可以统一渠道、身份、会话和调度，使个人助理不依赖一个前台应用常驻。代价是攻击面扩大，需要更严格的账户隔离、插件审核和凭据管理。

第四部分

框架、运行时与生产级治理

Agent 框架与协议的架构定位

33.1 不要把框架当作架构

框架提供 Agent、工具、状态、编排和部署原语，但不会替你决定任务边界、验证器和安全策略。同一框架可实现完全不同的架构；同一架构也可用不同框架实现。

33.2 OpenAI Agents SDK

OpenAI Agents SDK 提供 Agent、Tools、Handoffs、Guardrails、Sessions 和 Tracing 等原语 [70, 71, 72, 73]。适合轻量单/多 Agent、Handoff 和服务内编排。复杂耐久 workflow 仍需外部状态和任务系统。

33.3 Claude Agent SDK

Claude Agent SDK 公开了与 Claude Code 相近的工具循环、会话、Hooks、Skills、Sub-agents、MCP 和记忆加载能力 [41]。适合构建以 Coding/Computer-use 为核心、需要 Claude Code 扩展机制的 Agent。

33.4 LangGraph

LangGraph 定位为状态化图编排运行时，重点是持久化、耐久执行、人类中断、流式和子图 [74, 75, 77]。适合复杂控制流和长任务，但要求开发者显式建模状态。

33.5 Google ADK

ADK 支持 Agent 层次、顺序/并行 workflow、多 Agent、Session/State/Memory 和 A2A；ADK 2.0 进一步转向图 workflow 运行时 [78, 79, 80, 81]。适合 Google 生态和企业级多 Agent。

33.6 AutoGen

AutoGen 提供 AgentChat 与事件驱动 Core, 内置 RoundRobin、SelectorGroupChat、Swarm、Magnetic-One 和 GraphFlow 等团队模式 [32, 33]. 适合研究多 Agent 交互和复杂会话拓扑。

33.7 CrewAI

CrewAI 将 Crews 作为协作单元、Flows 作为结构化事件驱动骨架 [82, 83]. 这种 “Flow 管控制、Crew 管协作” 的分层适合业务流程型 Agent。

33.8 MCP 与 A2A

MCP 标准化 Agent/LLM 应用与工具、数据源的连接; A2A 面向 Agent 与 Agent 之间的跨框架通信 [84, 85]. 两者互补: MCP 让 Agent 使用能力, A2A 让独立 Agent 协作。

33.8.1 协议不等于信任

协议解决互操作, 不自动解决身份、授权、来源可信和 Prompt Injection。每个远程工具或 Agent 都应经过认证、能力限制、输入输出验证和审计。

33.9 框架选型表

框架/协议	最突出的架构能力	适合场景	主要注意点
OpenAI Agents SDK	Tools、Handoff、Guardrails、Tracing	轻量服务内 Agent、多 Agent 转交	耐久工作流需外接
Claude Agent SDK	Claude Code 式 Loop、Hooks、Skills、Sub-agents	Coding/Computer-use、可扩展 Agent	权限与环境治理
LangGraph	显式图、Checkpoint、Interrupt	长任务、复杂状态、审批	状态建模成本
Google ADK	多 Agent、模板/图工作流、A2A	企业与跨 Agent 生态	版本与运行时选择
AutoGen	多种 Group Chat/Swarm/Magnetic-One	多 Agent 研究与动态协作	群聊上下文和终止
CrewAI	Flow + Crew	业务流程与角色协作	角色不能替代工件契约

框架/协议	最突出的架构能力	适合场景	主要注意点
MCP	工具/数据接入标准	跨产品能力复用	服务端信任与权限
A2A	Agent 间互操作	跨框架、跨组织协作	身份、任务状态和安全

长任务、队列、检查点与故障恢复

34.1 任务持久化

长任务必须有独立 Task Record，包含目标、状态、计划版本、工件、预算、所有者、审批和最后心跳。进程与会话可以消失，Task Record 不能消失。

34.2 队列与 Worker

请求进入队列，由 Worker 获取租约执行。租约超时后可重新调度。并发控制按用户、Agent、工具和资源设置。模型调用只是 Worker 的一个步骤，不应阻塞整个调度系统。

34.3 检查点

在高成本计算后、外部副作用前后、人工暂停前保存检查点。检查点应包含状态快照和工件引用，不必保存全部模型内部文本。恢复时先验证外部世界是否变化。

34.4 幂等与副作用

每个写操作携带 Idempotency Key；重复执行返回已有结果。无法幂等的动作使用事务或补偿流程。Agent 的“我记得已经发过”不能替代服务端幂等。

34.5 重试策略

只对暂时性错误自动重试，采用指数退避和抖动。参数错误、权限错误和业务拒绝不应盲目重试。模型修复重试应限制次数并记录每次变化。

34.6 取消与超时

取消需要传播到子 Agent、工具和后台进程。超时分为单工具、单步骤、整个任务和静默等待。超时后应标记不确定状态，不能假定外部操作没有发生。

34.7 跨上下文窗口连续性

每个会话结束时生成结构化交接：已完成、当前状态、证据、未解决问题、下一步和禁止重复动作。下一会话从工件恢复，而不是只依赖压缩摘要。

Agent 安全架构与威胁模型

35.1 核心风险

Agent 能读取私有数据、调用工具并改变环境，因此 Prompt Injection、工具投毒、越权、凭据泄露、恶意文件、供应链插件和自动化误操作都会产生真实影响。

35.2 零信任分层

1. **输入层**：用户、网页、文档和工具返回均视为不受信任；
2. **模型层**：模型建议不等于授权；
3. **策略层**：根据用户、任务和动作做权限决策；
4. **执行层**：沙箱、网络出口、文件和进程隔离；
5. **数据层**：最小化上下文、脱敏、密钥代理；
6. **审计层**：记录工具、参数、结果、审批和身份。

35.3 Prompt Injection 防护

系统指令、用户目标、外部内容和工具元数据必须分层标记。来自网页或仓库的文本不能修改权限和系统规则。对敏感动作，策略引擎依据结构化意图和用户授权判断，不读取外部内容中的“授权声明”。

35.4 工具安全

工具采用能力模型：Agent 只拿到当前任务需要的有限能力。参数服务器端验证；高风险工具支持 dry-run；返回结果限制大小并净化控制字符。MCP 等协议提供互操作，但服务器仍需信任审查 [84]。

35.5 凭据与身份

模型不直接看到长期密钥。执行服务根据用户/Agent 身份换取短期令牌，并保留主体链。多 Agent 委派时必须传播最小权限，不能让子 Agent 自动继承 Supervisor 的全部凭据。

35.6 沙箱

沙箱限制文件、网络、系统调用、进程、资源和设备。构建代码、浏览网页、运行 Notebook 和分析附件应在不同风险域。沙箱逃逸和工具服务漏洞仍需传统安全防护。

35.7 审批设计

审批界面展示动作、参数、目标资源、数据流、不可逆性、证据和替代方案。批量审批必须明确范围和有效期。模型生成的自然语言摘要不能隐藏真实参数。

35.8 安全验证

使用攻击测试覆盖间接 Prompt Injection、混淆编码、工具返回投毒、权限边界、跨用户记忆泄露、重复执行和长任务劫持。安全评测应看完整轨迹，而不只看最终文本。

评测、Tracing 与调试

36.1 三层评测

第一层是任务结果，如成功率和质量；第二层是轨迹，如工具选择、步骤数、恢复和权限；第三层是系统指标，如延迟、成本、稳定性和资源消耗。只看最终成功率无法解释架构改动。

36.2 结果指标

根据任务使用编译/测试通过率、引用支持率、SQL 正确率、业务完成率、人工偏好和安全违规率。需要区分“任务完成”“结果正确”和“过程合规”。

36.3 轨迹指标

记录无效工具率、重复动作率、计划修改次数、验证覆盖、Handoff 次数、上下文长度、失败恢复率和人工介入点。多 Agent 还要测消息冗余和并行效率。

36.4 Tracing

Trace 应关联用户请求、模型调用、工具、子 Agent、状态节点、审批和工作。OpenAI Agents SDK 等框架提供内置 Tracing 原语 [72]。敏感参数需脱敏，但不能因此失去可调试性。

36.5 失败分类

推荐至少分类：任务理解、上下文检索、计划、工具选择、参数、环境、验证、权限、状态一致性、终止和用户交互。每类失败对应不同修复手段，不能统一归因于“模型不够强”。

36.6 消融实验

验证架构模块时，应移除或替换单个组件：无记忆、无规划、无 Critic、无 Subagent、无并行、无项目说明、不同 ACI。保持模型和数据一致，才能判断收益来自哪里。

36.7 回放与确定性

保存工具返回快照和模型请求元数据，使失败可回放。外部环境变化时，可用 Mock 或录制结果复现控制逻辑。对随机模型应运行多次并报告方差。

36.8 Agent Evals 的特性

长轨迹评测需要任务级 Rubric、环境重置、稳定判据和失败证据。Anthropic 的 Agent Eval 实践强调多轮工具行为使评测比普通模型问答更复杂 [23]。

架构选型案例与设计审查

37.1 案例一：内部知识问答

若只需回答有来源的问题，采用检索-生成-引用验证 workflow 即可。只有当问题需要跨源迭代、调用计算或主动澄清时才引入 ReAct。无需多 Agent。

37.2 案例二：自动修复代码 Issue

外层使用 FSM 管理勘察、复现、修改、验证和交付；局部使用 Coding ReAct；仓库上下文采用符号检索/Repo Map；执行在 Worktree 与容器中；测试作为硬 Verifier。大 Issue 可让 Subagent 并行搜索，但最终写入串行合并。

37.3 案例三：跨来源研究报告

Lead Agent 生成问题树，多个 Search Worker 并行收集证据，Evidence Ledger 共享工件，Synthesizer 写作，Citation Verifier 检查。适合 Orchestrator-Workers + Blackboard，而不是自由群聊。

37.4 案例四：企业客服与业务办理

Router 选择领域；解释型问题走 RAG；交易型问题走 FSM/HTN；高风险动作审批；Handoff 给人工。长期用户偏好与业务事实分离，所有工具按用户身份授权。

37.5 案例五：持续安全漏洞分析

Supervisor 分配静态、动态和验证任务；黑板保存攻击面、候选和证据；沙箱运行 PoC；Verifier 使用可复现判据；失败反思进入带来源的 Episodic Memory。自动对外报告需要人工审批。

37.6 案例六：个人长期助理

Gateway 负责渠道与唤醒，BDI 管理长期目标，事件驱动任务系统负责定时与条件触发，记忆中心保存偏好，工具使用最小权限。通知策略和去重与模型分离。

37.7 设计审查的十二个问题

1. 任务的输入、输出和成功判据是什么？
2. 为什么需要 Agent，而不是一次调用或固定 workflow？
3. 控制权由模型、代码还是人类掌握？
4. 状态的唯一事实来源在哪里？
5. 哪些信息进入上下文，哪些进入记忆？
6. 工具的副作用、权限和错误语义是什么？
7. 是否存在确定性验证器？
8. 任务如何暂停、恢复、取消和重放？
9. 多 Agent 是否带来真实并行或专业差异？
10. 外部内容如何防止 Prompt Injection？
11. 如何追踪成本、延迟和失败原因？
12. 最坏情况下 Agent 能造成什么影响，边界是否足够硬？

关键结论

推荐的设计顺序是：任务契约 → 状态与成功判据 → 工具和权限 → 验证器 → 控制流 → 模型策略 → 记忆 → 多 Agent。把多 Agent 放在最后，而不是最先。

架构术语速查

术语	含义
Agent Loop	模型选择动作、环境执行并返回观测的闭环。
Workflow	由代码预定义控制流的步骤组合，可包含模型节点。
Orchestrator	负责拆分、调度、监控和汇总多个 Worker 的控制 Agent。
Handoff	当前 Agent 将对话或任务所有权转交另一 Agent。
Agent-as-Tool	上层 Agent 调用下层 Agent，但保留最终控制权。
Blackboard	多个专家读写的共享结构化工作空间。
ACI	为 Agent 设计的计算机/工具交互接口。
Checkpoint	可用于恢复任务的持久化状态快照。
Guardrail	对输入、输出、工具或策略施加的约束与检查。
Skill	可复用的程序性说明、资源和脚本包。
MCP	Agent/LLM 应用与工具、数据源之间的开放连接协议。
A2A	独立 Agent 之间通信和协作的开放协议。

架构模式总表

架构模式	最适合的问题	核心优点	主要风险
反应式	高频、条件明确的动作	快速、可预测	无长程规划
FSM	阶段和异常路径明确	可审计、易测试	状态爆炸
行为树	持续反应与回退	模块化、可复用	全局状态复杂
BDI	多目标长期 Agent	承诺与目标清楚	信念维护成本
HTN	领域流程与正确顺序	可靠、搜索空间小	建模成本高
ReAct	开放探索与动态反馈	灵活、闭环	循环和成本
CodeAct	复杂工具组合	表达力强、减少往返	沙箱风险
Plan-and-Execute	中长任务	全局计划、可并行	初始假设错误
ReWOO	可预见依赖的多步检索	Token/延迟效率	适应未知观测弱
LLMCompiler / DAG	工具依赖与并行	高吞吐、可调度	依赖判断错误
ToT/LATS	候选多、可回滚	能探索替代路径	成本高、评价难
Self-Refine	可迭代改善的产物	简单通用	自评偏差
CRITIC	可用工具验证的输出	证据驱动纠错	验证工具质量
MemGPT/记忆中心	长会话与跨会话	持久连续性	记忆污染
图/耐久工作流	长任务、审批、恢复	状态清楚、可恢复	建模复杂
Router-Specialist	多领域分流	边界清晰、成本低	路由错误
Handoff/Swarm	动态服务所有权	灵活、去中心化	环路与责任不清
Supervisor-Workers	可拆分并行任务	统一协调、专业化	Supervisor 瓶颈
Debate / Committee	需要多候选与反例	提高多样性	错误相关、成本高
SOP/流水线	有组织流程和工件	阶段明确、可审计	错误级联
Blackboard	多专家共享证据	异步、低重复	Schema 和冲突管理

进一步阅读路线

C.1 经典控制与规划

建议依次阅读 BDI、行为树、HTN 和黑板架构，理解目标承诺、反应控制、任务分解和共享工作空间。

C.2 LLM 单 Agent

阅读 ReAct、CodeAct、ReWOO、LLMCompiler、LATS、Self-Refine、Reflexion、CRITIC、MemGPT 和 Voyager，建立“行动、规划、搜索、验证、记忆”的完整谱系。

C.3 多 Agent

阅读 AutoGen、CAMEL、MetaGPT、ChatDev、Magentic-One，并重点比较消息拓扑、共享状态、工件契约、终止和验证，而不是只记角色名称。

C.4 Coding Agent

阅读 SWE-agent、OpenHands、Agentless、MASAI 与 Aider 文档，再结合 Claude Code 和 Codex 的官方机制，关注 ACI、仓库上下文、环境隔离和测试闭环。

参考资料

- [1] Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. <https://arxiv.org/abs/2210.03629>
- [2] Karpas, E. et al. *MRKL Systems: A Modular, Neuro-Symbolic Architecture That Combines Large Language Models, External Knowledge Sources and Discrete Reasoning*. 2022. <https://arxiv.org/abs/2205.00445>
- [3] Wang, X. et al. *Executable Code Actions Elicit Better LLM Agents*. ICML 2024. <https://arxiv.org/abs/2402.01030>
- [4] Xu, B. et al. *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*. 2023. <https://arxiv.org/abs/2305.18323>
- [5] Kim, S. et al. *An LLM Compiler for Parallel Function Calling*. ICML 2024. <https://arxiv.org/abs/2312.04511>
- [6] Yao, S. et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. <https://arxiv.org/abs/2305.10601>
- [7] Zhou, A. et al. *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*. ICML 2024. <https://arxiv.org/abs/2310.04406>
- [8] Madaan, A. et al. *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS 2023. <https://arxiv.org/abs/2303.17651>
- [9] Shinn, N. et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023. <https://arxiv.org/abs/2303.11366>
- [10] Gou, Z. et al. *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*. 2023. <https://arxiv.org/abs/2305.11738>
- [11] Packer, C. et al. *MemGPT: Towards LLMs as Operating Systems*. 2023. <https://arxiv.org/abs/2310.08560>
- [12] Park, J. S. et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST 2023. <https://arxiv.org/abs/2304.03442>
- [13] Wang, G. et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. <https://arxiv.org/abs/2305.16291>
- [14] Rao, A. S.; Georgeff, M. P. *BDI Agents: From Theory to Practice*. ICMAS 1995. <https://cdn.aaai.org/ICMAS/1995/ICMAS95-042.pdf>

- [15] Georgeff, M. et al. *The Belief-Desire-Intention Model of Agency*. 1998. <https://www.cs.ox.ac.uk/people/michael.wooldridge/pubs/atal98b.pdf>
- [16] Iovino, M. et al. *A Survey of Behavior Trees in Robotics and AI*. 2022. <https://arxiv.org/abs/2005.05842>
- [17] Colledanchise, M.; Ögren, P. *Behavior Trees in Robotics and AI: An Introduction*. 2017. <https://arxiv.org/abs/1709.00084>
- [18] Georgievski, I.; Aiello, M. *An Overview of Hierarchical Task Network Planning*. 2014. <https://arxiv.org/abs/1403.7426>
- [19] Hayes-Roth, B. *A Blackboard Architecture for Control*. Artificial Intelligence, 1985. <https://www.sciencedirect.com/science/article/pii/0004370285900633>
- [20] Anthropic. *Building Effective AI Agents*. <https://www.anthropic.com/engineering/building-effective-agents>
- [21] Anthropic. *How We Built Our Multi-Agent Research System*. <https://www.anthropic.com/engineering/multi-agent-research-system>
- [22] Anthropic. *Effective Harnesses for Long-Running Agents*. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- [23] Anthropic. *Demystifying Evals for AI Agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [24] Wu, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. <https://arxiv.org/abs/2308.08155>
- [25] Li, G. et al. *CAMEL: Communicative Agents for Mind Exploration of Large Scale Language Model Society*. NeurIPS 2023. <https://arxiv.org/abs/2303.17760>
- [26] Hong, S. et al. *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*. 2023. <https://arxiv.org/abs/2308.00352>
- [27] Qian, C. et al. *ChatDev: Communicative Agents for Software Development*. ACL 2024. <https://arxiv.org/abs/2307.07924>
- [28] Fournay, A. et al. *Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks*. 2024. <https://arxiv.org/abs/2411.04468>
- [29] Microsoft. *Magentic-One Documentation*. <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/magentic-one.html>
- [30] Microsoft. *AutoGen Swarm*. <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/swarm.html>

- [31] Microsoft. *AutoGen Selector Group Chat*. <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/selector-group-chat.html>
- [32] Microsoft. *AutoGen AgentChat*. <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/>
- [33] Microsoft. *AutoGen Teams*. <https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/tutorial/teams.html>
- [34] Yang, J. et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. <https://arxiv.org/abs/2405.15793>
- [35] Wang, X. et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. ICLR 2025. <https://arxiv.org/abs/2407.16741>
- [36] Wang, X. et al. *The OpenHands Software Agent SDK*. 2025. <https://arxiv.org/abs/2511.03690>
- [37] Aider. *Repository Map*. <https://aider.chat/docs/repomap.html>
- [38] Xia, C. S. et al. *Agentless: Demystifying LLM-based Software Engineering Agents*. FSE 2025. <https://arxiv.org/abs/2407.01489>
- [39] Arora, D. et al. *MASAI: Modular Architecture for Software-engineering AI Agents*. 2024. <https://arxiv.org/abs/2406.11638>
- [40] Anthropic. *How Claude Code Works*. <https://code.claude.com/docs/en/how-claude-code-works>
- [41] Anthropic. *Claude Agent SDK Overview*. <https://code.claude.com/docs/en/agent-sdk/overview>
- [42] Anthropic. *Claude Code Subagents*. <https://code.claude.com/docs/en/sub-agents>
- [43] Anthropic. *Claude Code Hooks Reference*. <https://code.claude.com/docs/en/hooks>
- [44] Anthropic. *Connect Claude Code to Tools via MCP*. <https://code.claude.com/docs/en/mcp>
- [45] Anthropic. *How Claude Remembers Your Project*. <https://code.claude.com/docs/en/memory>
- [46] Anthropic. *The .claude Directory*. <https://code.claude.com/docs/en/claude-directory>
- [47] Anthropic. *Claude Code Security*. <https://code.claude.com/docs/en/security>
- [48] OpenAI. *Codex Overview*. <https://developers.openai.com/codex/overview/>

- [49] OpenAI. *Codex CLI*. <https://developers.openai.com/codex/cli>
- [50] OpenAI. *Custom Instructions with AGENTS.md*. <https://developers.openai.com/codex/agent-configuration/agents-md>
- [51] OpenAI. *Build Skills for Codex*. <https://developers.openai.com/codex/build-skills>
- [52] OpenAI. *Codex MCP Server*. <https://developers.openai.com/codex/mcp-server>
- [53] OpenAI. *Codex Subagents*. <https://developers.openai.com/codex/subagents>
- [54] OpenAI. *Codex Sandbox*. <https://developers.openai.com/codex/sandboxing>
- [55] OpenAI. *Codex Best Practices*. <https://developers.openai.com/codex/learn/best-practices>
- [56] OpenAI. *Codex Automations*. <https://developers.openai.com/codex/automations>
- [57] OpenAI. *Run Long-Horizon Tasks with Codex*. <https://developers.openai.com/blog/run-long-horizon-tasks-with-codex>
- [58] OpenAI. *Introducing the Codex App*. <https://openai.com/index/introducing-the-codex-app/>
- [59] OpenClaw. *OpenClaw Documentation*. <https://docs.openclaw.ai/>
- [60] OpenClaw. *Gateway Architecture*. <https://docs.openclaw.ai/concepts/architecture>
- [61] OpenClaw. *Agent Loop*. <https://docs.openclaw.ai/concepts/agent-loop>
- [62] OpenClaw. *Memory Overview*. <https://docs.openclaw.ai/concepts/memory>
- [63] OpenClaw. *Agent Workspace*. <https://docs.openclaw.ai/concepts/agent-workspace>
- [64] OpenClaw. *Skills*. <https://docs.openclaw.ai/tools/skills>
- [65] OpenClaw. *Sub-agents*. <https://docs.openclaw.ai/tools/subagents>
- [66] OpenClaw. *Multi-agent Routing*. <https://docs.openclaw.ai/concepts/multi-agent>
- [67] OpenClaw. *Delegate Architecture*. <https://docs.openclaw.ai/concepts/delegate-architecture>
- [68] OpenClaw. *Automations*. <https://docs.openclaw.ai/automation/cron-jobs>
- [69] OpenClaw. *Agent Runtime Architecture*. <https://docs.openclaw.ai/agent-runtime-architecture>

- [70] OpenAI. *OpenAI Agents SDK*. <https://openai.github.io/openai-agents-python/>
- [71] OpenAI. *Handoffs*. <https://openai.github.io/openai-agents-python/handoffs/>
- [72] OpenAI. *Tracing*. <https://openai.github.io/openai-agents-python/tracing/>
- [73] OpenAI. *Guardrails*. <https://openai.github.io/openai-agents-python/guardrails/>
- [74] LangChain. *LangGraph Overview*. <https://docs.langchain.com/oss/python/langgraph/overview>
- [75] LangChain. *LangGraph Persistence*. <https://docs.langchain.com/oss/python/langgraph/persistence>
- [76] LangChain. *LangGraph Interrupts*. <https://docs.langchain.com/oss/python/langgraph/interrupts>
- [77] LangChain. *LangGraph Subgraphs*. <https://docs.langchain.com/oss/python/langgraph/use-subgraphs>
- [78] Google. *Agent Development Kit*. <https://google.github.io/adk-docs/>
- [79] Google. *ADK Multi-agent Workflows*. <https://google.github.io/adk-docs/agents/multi-agents/>
- [80] Google. *ADK Parallel Workflow*. <https://google.github.io/adk-docs/agents/workflow-agents/parallel-agents/>
- [81] Google. *ADK 2.0 Workflow Runtime*. <https://google.github.io/adk-docs/2.0/>
- [82] CrewAI. *CrewAI Documentation*. <https://docs.crewai.com/>
- [83] CrewAI. *CrewAI Flows*. <https://docs.crewai.com/en/concepts/flows>
- [84] Model Context Protocol. *MCP Specification*. <https://modelcontextprotocol.io/specification/2025-06-18>
- [85] Agent2Agent Protocol. *A2A Protocol Specification*. <https://a2a-protocol.org/latest/>